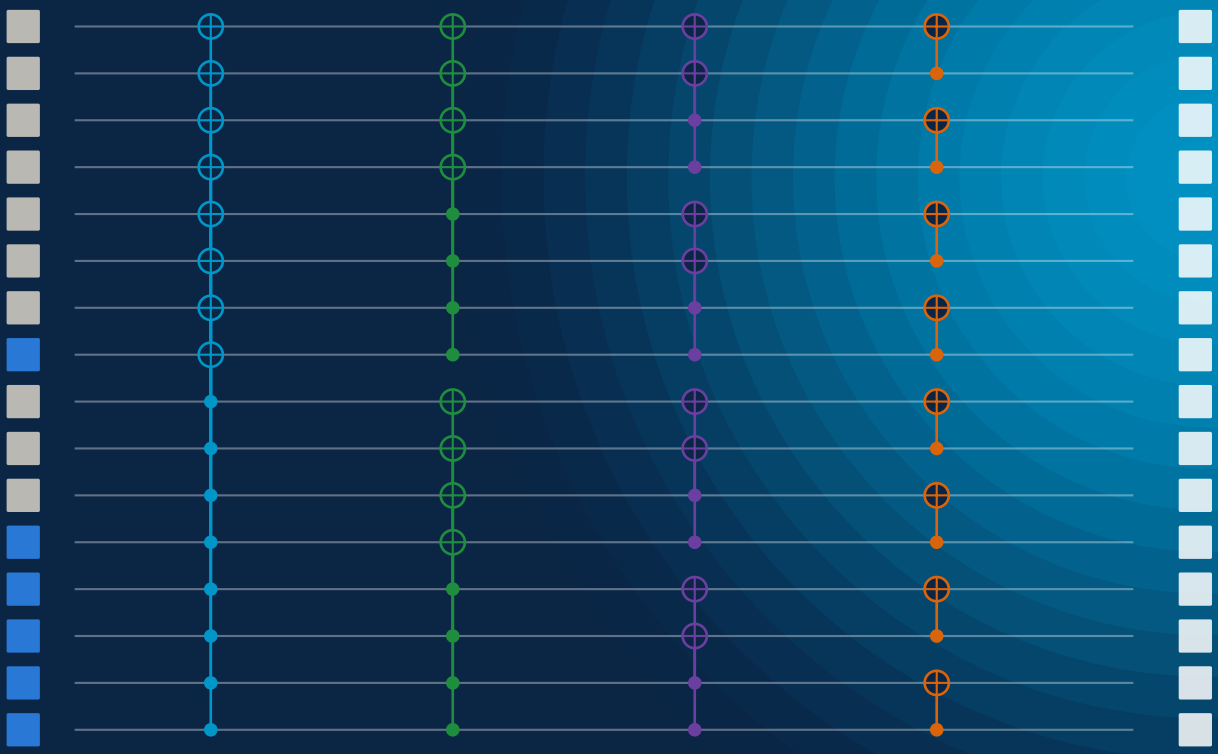


A COMPLETE VISUAL AND MATHEMATICAL GUIDE

# POLAR CODES

## From Zero to Expert

Channel polarization • Encoding • SC, SCL and CRC-aided decoding • 5G NR • Tested Python simulations • 15 chapters, solved problems and a final project

**Mahmoud Abdelaziz**

Military Technical College, Cairo

First edition • 2026

**Polar Codes: From Zero to Expert — A Complete Visual and Mathematical Guide**

First edition, October 2026.

All simulation results in this book were produced with the Python code printed in Chapter 11 and Chapter 15 (NumPy, fixed random seeds). They are Monte-Carlo estimates, not theoretical bounds; re-running the code reproduces them. Specification details of 3GPP NR are summarised for teaching; always consult the current release of TS 38.212 for implementation work.

Typeset with XeLaTeX. Figures drawn with Ti*k*Z and Matplotlib.

# How to Use This Book

This book takes you from *zero* — you do not need to know what a channel code is — to the level of an engineer who can design, simulate and explain the polar codes used in 5G. Every topic is taught in the same seven steps:




## The colour code of the boxes

**Definition 0.1 • Definitions are cyan**
A precise statement of a new term. Learn these by heart.

**Intuition**
Gold boxes explain the idea in everyday words, before any mathematics.

**Worked Example 0.1 • Worked examples are green**
Real numbers, every step shown. Re-do them with a pencil.

**Advanced • Purple is for advanced material**
You may skip purple boxes on a first reading.

**Important note**
Orange boxes hold facts that are easy to forget but important in practice.

**! Common mistake**
Red boxes show errors that students (and engineers!) really make.

**Professional insight**
Grey boxes connect the theory to real systems and current research.

## Conventions used in the whole book

Polar-code books differ in small conventions, and mixing them is the most common source of confusion. This book fixes them once:

| | |
|---|---|
| **Kernel** | $\mathbf{F} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$ |
| **Generator matrix** | $\mathbf{G}_N = \mathbf{F}^{\otimes n}$, $N = 2^n$, *without* bit-reversal (same as 3GPP NR) |
| **Encoding** | row vectors over GF(2): $\mathbf{x} = \mathbf{u}\,\mathbf{G}_N$ |
| **Indices** | bits are numbered $0, 1, \ldots, N-1$ in natural order |
| **Information / frozen sets** | $\mathcal{A}$ (size $K$) and $\mathcal{F}$ (size $N-K$); frozen bits are 0 |
| **BPSK** | bit $0 \mapsto +1$, bit $1 \mapsto -1$; symbol energy $E_s = 1$ |
| **LLR** | $L = \ln \dfrac{P(\text{bit}=0 \mid y)}{P(\text{bit}=1 \mid y)}$; positive favours 0 |
| **Noise** | real AWGN, variance $\sigma^2 = N_0/2$, $E_s/N_0 = R\,E_b/N_0$ |
| **Colours of bits** | 1 information 0 frozen 1 CRC 1 coded |

## The software

All Python code is in one small library, `polar.py`, printed in full in Chapter 11. It was tested against brute-force calculations (the SC decoder reproduces the exact bit-channel LLRs of Arıkan's definition, and the list decoder with a large list returns the maximum-likelihood codeword). All plots of error rates are Monte-Carlo results produced by that code with fixed random seeds, so you can reproduce every curve.

# Contents


**How to Use This Book** . . . . . iii

## I Beginner Level: Digital Communication Foundations

**1 Introduction to Digital Communication** . . . . . 2

1.1 What is digital communication? . . . . . 2
1.2 Transmitter, channel and receiver . . . . . 3
1.3 What causes transmission errors? . . . . . 3
1.4 Bit error rate (BER) . . . . . 4
1.5 Block error rate (BLER) . . . . . 6
1.6 Why error-correcting codes are necessary. . . . . 7
1.7 Simple real-life examples . . . . . 7
1.8 Chapter summary . . . . . 7

**2 Fundamentals of Channel Coding** . . . . . 10

2.1 What is channel coding?. . . . . 10
2.2 Codewords, block codes and code rate . . . . . 11
2.3 Hamming distance . . . . . 11
2.4 Linear block codes and generator matrices . . . . . 12
2.5 Three channel models . . . . . 13
2.6 Shannon channel capacity . . . . . 13
2.7 Why polar codes are important . . . . . 15
2.8 Chapter summary . . . . . 15

## II Intermediate Level: Understanding Polar Codes

**3 Introduction to Polar Codes** . . . . . 19

3.1 What are polar codes? . . . . . 19
3.2 History: Arıkan's 2009 breakthrough . . . . . 20

3.3 Why were polar codes invented? . . . 20
3.4 Channel polarization explained intuitively . . . 21
3.5 Reliable and unreliable bit-channels . . . 21
3.6 Frozen bits and information bits . . . 22
3.7 Block length, information length and code rate . . . 22
3.8 Capacity-achieving behaviour . . . 23
3.9 Polar codes versus LDPC and turbo codes . . . 23
3.10 Chapter summary . . . 24

**4 Channel Polarization — The Core Idea . . . 26**

4.1 Channel combining . . . 26
4.2 Channel splitting . . . 27
4.3 The binary erasure channel: an exact example . . . 28
4.4 Visualizing polarization for N = 2, 4, 8, 16 . . . 29
4.5 The Bhattacharyya parameter . . . 30
4.6 Why do channels polarize? . . . 31
4.7 Chapter summary . . . 32

**5 Polar Code Encoding . . . 35**

5.1 The polar encoding kernel . . . 35
5.2 The Kronecker product . . . 36
5.3 Frozen-bit placement and information-bit selection . . . 36
5.4 Recursive encoding . . . 37
5.5 Encoding stages and butterfly diagrams . . . 37
5.6 Encoding complexity . . . 39
5.7 Chapter summary . . . 39

**III Advanced Level: Decoding Algorithms**

**6 Successive Cancellation (SC) Decoding . . . 43**

6.1 Why decoding is necessary . . . 43
6.2 The successive cancellation concept . . . 43
6.3 Likelihood ratios and log-likelihood ratios . . . 44
6.4 The f and g functions . . . 45
6.5 The SC decoding tree . . . 46
6.6 Hard decisions and frozen-bit handling . . . 46
6.7 Step-by-step numerical decoding example . . . 47
6.8 A simple SC decoder in Python . . . 48

6.9 Computational complexity . . . 49
6.10 Advantages and limitations . . . 49
6.11 Chapter summary . . . 50

**7 Successive Cancellation List (SCL) Decoding . . . 52**

7.1 Why SC decoding is sometimes insufficient . . . 52
7.2 Path splitting . . . 53
7.3 Path metrics . . . 53
7.4 Candidate pruning . . . 54
7.5 Worked SCL decoding example . . . 54
7.6 SCL versus SC performance . . . 55
7.7 Complexity and memory requirements . . . 56
7.8 Chapter summary . . . 57

**8 CRC-Aided SCL Decoding . . . 59**

8.1 What is a CRC? . . . 59
8.2 How a CRC helps polar decoding . . . 60
8.3 CRC attachment . . . 61
8.4 CRC-based candidate selection . . . 61
8.5 Complete transmitter and receiver workflow . . . 61
8.6 Practical examples . . . 62
8.7 The price of the CRC . . . 63
8.8 Chapter summary . . . 63

**9 Additional Professional Topics . . . 66**

9.1 Systematic polar encoding . . . 66
9.2 Belief propagation (BP) decoding . . . 67
9.3 SCAN decoding . . . 67
9.4 Fast-SSC decoding . . . 68
9.5 Adaptive SCL decoding . . . 68
9.6 Polar-code construction methods . . . 69
9.7 Puncturing, shortening and repetition . . . 70
9.8 Rate matching . . . 71
9.9 Finite-length performance . . . 71
9.10 Hardware implementation considerations . . . 72
9.11 Chapter summary . . . 72

**IV Professional Level: 5G and Practical Implementation**

10 Stub 10 . . . . . 76
11 Stub 11. . . . . 77
12 Stub 12 . . . . . 78

V Expert Practice

13 Stub 13 . . . . . 80
14 Stub 14 . . . . . 81
15 Stub 15 . . . . . 82

PART I

# Beginner Level: Digital Communication Foundations

Bits, channels, noise and errors. Why we must add redundancy, and how much the channel allows. No prior knowledge is assumed.

CHAPTER

# 1



# Introduction to Digital Communication

**IN THIS CHAPTER YOU WILL LEARN**

- what a digital communication system is;
- the roles of transmitter, channel and receiver;
- why bits arrive with errors;
- the AWGN channel and BPSK modulation;
- how to compute the bit error rate $Q(\sqrt{2E_b/N_0})$;
- BER and BLER, and how they are related;
- why error-correcting codes are unavoidable.

## 1.1 What is digital communication?

**Intuition**

Imagine you must tell a friend across a noisy football stadium whether to meet at Gate A or Gate B. You agree beforehand: one raised arm means A, two raised arms means B. You have turned your message into a small set of agreed *symbols*. Even if the view is blurred, your friend only has to decide between two possibilities — not guess a whole sentence. This is the core idea of *digital* communication: every message is first turned into a sequence of symbols from a small alphabet, usually the two symbols 0 and 1.

**Definition 1.1 • Digital communication**

**Digital communication** is the transfer of information as a sequence of symbols taken from a *finite* alphabet. In almost all modern systems the alphabet is binary, $\{0, 1\}$, and each symbol is called a **bit** (*binary digit*).

Voice, images and video are not naturally binary. A **source encoder** (for example a speech codec in a phone, or JPEG for images) converts them into bits and removes useless repetition. From then on, the communication system only sees bits and does not care what they mean. This separation is very powerful: the same 5G radio carries a voice call, a web page and a video without knowing the difference.

### 1.1.1 Why digital and not analogue?

- **Regeneration.** A noisy digital signal can be *decided* and re-created perfectly at each repeater; analogue noise accumulates.
- **Error control.** We can add clever redundancy and correct errors — the topic of this book.
- **Integration.** All kinds of data become one bit stream that can be encrypted, compressed, stored and processed by cheap digital chips.

## 1.2 Transmitter, channel and receiver

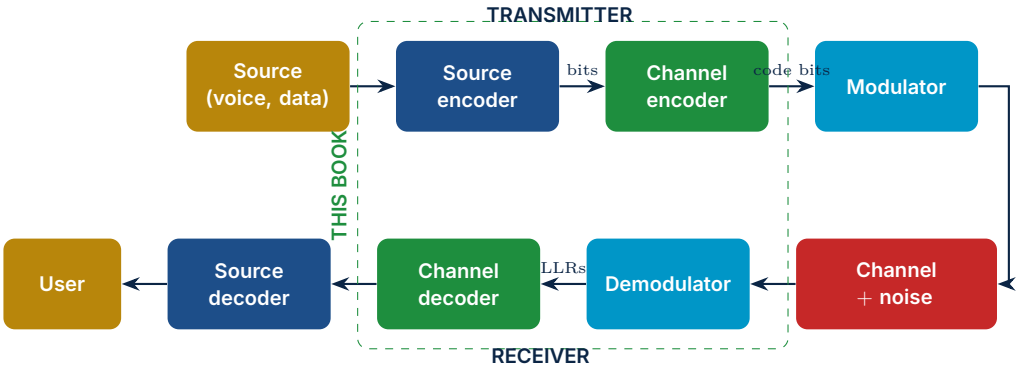



**Figure 1.1.** Block diagram of a digital communication system. Polar codes live in the green blocks.

Fig. 1.1 shows the classic chain proposed by Shannon in 1948.

- The **transmitter** prepares the bits for the physical medium. The **channel encoder** adds controlled redundancy; the **modulator** turns bits into a physical waveform (a voltage, a light pulse, a radio wave).
- The **channel** is everything between the transmitter and the receiver: copper, optical fibre, air, the electronics of the radio, even a hard disk (storage is communication *in time*).
- The **receiver** undoes the transmitter operations. The **demodulator** measures the waveform and gives the decoder a *soft* number per bit (we will call it an LLR); the **channel decoder** uses the redundancy to find and correct errors.

## 1.3 What causes transmission errors?

A bit error happens when the receiver decides 1 while 0 was sent, or the opposite. The physical causes are many:

| | |
|---|---|
| **Thermal noise** | random motion of electrons in every resistor and amplifier; always present. |
| **Interference** | signals of other users, neighbouring cells, other systems in the same band. |
| **Fading** | the radio wave arrives over many paths that sometimes cancel each other. |
| **Distortion** | filters and amplifiers that are not ideal; inter-symbol interference. |
| **Synchronisation errors** | small mistakes in timing, frequency or phase. |

### 1.3.1 Noise and the AWGN model

For learning and for most code design we use one simple, very accurate model of thermal noise.

**Definition 1.2 • AWGN channel**

In the **additive white Gaussian noise** (AWGN) channel the received value is

$$y = s + w, \qquad w \sim \mathcal{N}(0, \sigma^2),$$

where $s$ is the transmitted amplitude and $w$ is a Gaussian random variable with mean 0 and variance $\sigma^2 = N_0/2$. *Additive*: the noise is added. *White*: noise samples at different times are independent. *Gaussian*: the noise has the bell-shaped probability density

$$p(w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{w^2}{2\sigma^2}\right).$$

The Gaussian shape is not a guess: thermal noise is the sum of a huge number of tiny independent effects, and the central limit theorem says such a sum is Gaussian.

### 1.3.2 BPSK: the simplest modulation

**Definition 1.3 • BPSK**

**Binary phase-shift keying** maps each bit to one amplitude: $0 \mapsto s = +1$, $1 \mapsto s = -1$. In formula form $s = 1 - 2c$ for code bit $c \in \{0, 1\}$. The energy per symbol is $E_s = s^2 = 1$.

The receiver sees $y = \pm 1 + w$. The natural decision is: say 0 if $y > 0$ and 1 if $y < 0$. Fig. 1.2 shows the two possible probability densities of $y$; the shaded tails are the error events.

## 1.4 Bit error rate (BER)

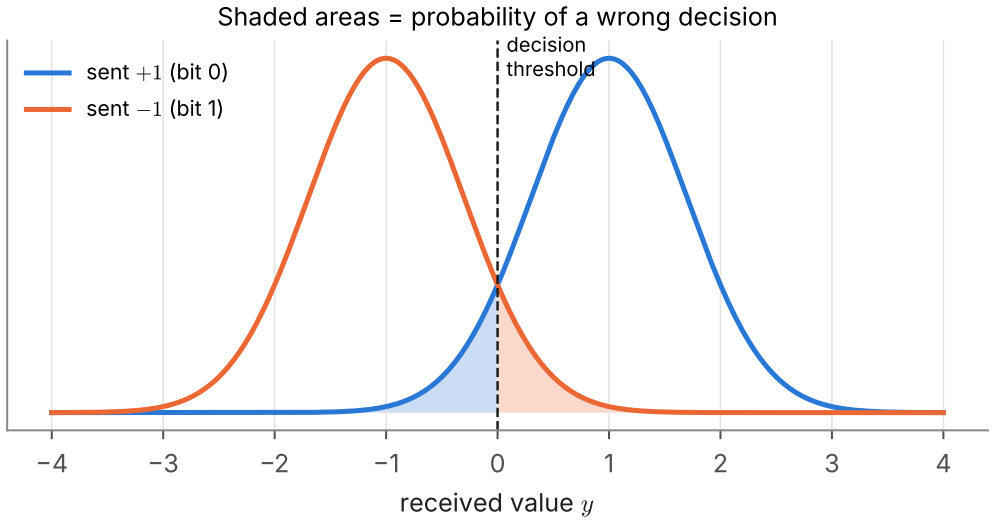



**Figure 1.2.** Densities of the received value for BPSK in AWGN ($\sigma = 0.7$).

### 1.4.1 Derivation of the BPSK error probability

Assume bit 0 was sent, so $y = 1 + w$. An error happens when $y < 0$, that is when $w < -1$:

$$\begin{aligned} P_b &= P(w < -1) = P(w > 1) \qquad \text{(the Gaussian is symmetric)} \\ &= \int_1^\infty \frac{1}{\sqrt{2\pi\sigma^2}} e^{-w^2/2\sigma^2}\, dw = \int_{1/\sigma}^\infty \frac{1}{\sqrt{2\pi}} e^{-t^2/2}\, dt \qquad (t = w/\sigma) \\ &= Q\left(\frac{1}{\sigma}\right). \end{aligned} \tag{1.1}$$

Here $Q(x) = \int_x^\infty \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt$ is the tail probability of a standard Gaussian. By symmetry the same result holds when 1 is sent.

Engineers prefer to write this with the **energy per information bit** $E_b$ and the noise level $N_0$. Without coding each symbol carries one bit, so $E_b = E_s = 1$, and $\sigma^2 = N_0/2$ gives $1/\sigma = \sqrt{2E_b/N_0}$:

$$P_b^{\text{BPSK}} = Q\left(\sqrt{\frac{2E_b}{N_0}}\right) \qquad \text{(uncoded BPSK over AWGN).}$$

The ratio $E_b/N_0$ is usually given in decibels: $(E_b/N_0)_{\text{dB}} = 10\log_{10}(E_b/N_0)$.

**Worked Example 1.1 • BER of uncoded BPSK**

**Question.** Find the BER of uncoded BPSK at $E_b/N_0 = 4\,\text{dB}$.
**Step 1 — leave decibels.** $E_b/N_0 = 10^{4/10} = 2.512$.
**Step 2 — argument of $Q$.** $\sqrt{2 \times 2.512} = \sqrt{5.024} = 2.241$.
**Step 3 — evaluate.** $Q(2.241) = 0.0125$. So about one bit in 80 is wrong.
Repeating the steps: $6.8\,\text{dB} \Rightarrow P_b \approx 9.9 \times 10^{-4}$ and $9.6\,\text{dB} \Rightarrow P_b \approx 9.7 \times 10^{-6}$. To divide the error rate by 100 we need almost 3 dB more power — about twice the power!

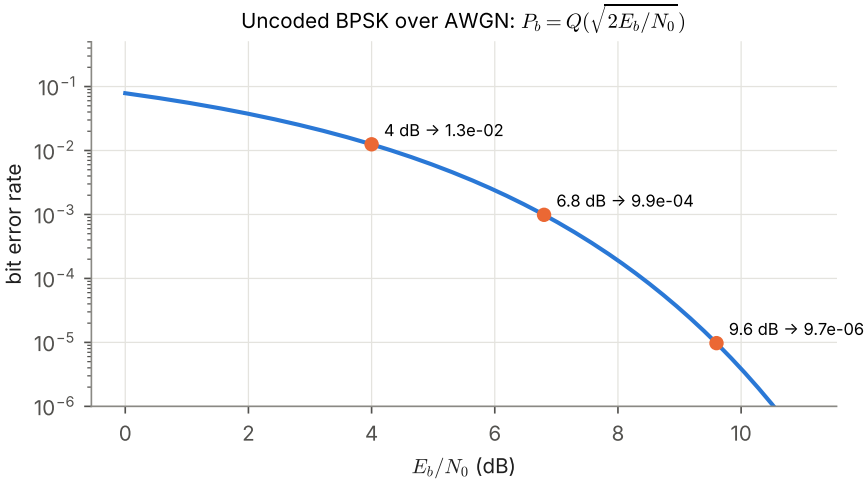



**Figure 1.3.** BER of uncoded BPSK. The curve falls slowly: power is an expensive way to buy reliability.

**Definition 1.4 • Bit error rate**

The **bit error rate** (BER) is the fraction of information bits that are decoded wrongly:

$$\text{BER} = \frac{\text{number of wrong bits}}{\text{number of transmitted bits}}.$$

In simulation we estimate it by counting; in theory it is the probability $P_b$.

## 1.5 Block error rate (BLER)

Real systems send data in *blocks* (packets, frames, transport blocks). One wrong bit is usually enough to make the whole block useless, because the block's checksum (CRC) fails and the block is thrown away or re-transmitted.

**Definition 1.5 • Block error rate**

The **block error rate** (BLER), also called frame error rate (FER), is

$$\text{BLER} = \frac{\text{number of blocks with at least one wrong bit}}{\text{number of transmitted blocks}}.$$

If $K$ bits are in error *independently* with probability $p$, the block is correct only if all $K$ bits are correct:

$$\text{BLER} = 1 - (1-p)^K \;\approx\; Kp \quad (\text{when } Kp \ll 1).$$

**Worked Example 1.2 • From BER to BLER**

A packet has $K = 1000$ bits and the raw BER is $p = 10^{-3}$.

$$\text{BLER} = 1 - (1 - 10^{-3})^{1000} = 1 - e^{1000\ln(0.999)} = 1 - e^{-1.0005} = 0.632.$$

Almost two packets out of three are lost, although only one bit in a thousand is wrong! With $K = 100$ the BLER is 0.095. This simple calculation is the strongest argument for error-correcting codes.

**BER or BLER?**

5G control channels and most modern standards specify targets as BLER (for example $10^{-2}$ or $10^{-3}$ for control information). Coded errors are *not* independent — a decoding failure usually breaks several bits together — so the formula $1-(1-p)^K$ is only for uncoded or independent errors. For coded systems we measure BLER directly.

## 1.6 Why error-correcting codes are necessary

There are only two ways to make communication more reliable:

1. **More power.** Fig. 1.3 shows this is expensive: from BER $10^{-3}$ to $10^{-5}$ costs about 2.8 dB (almost 2× power). Batteries, interference to other users and regulation all limit power.
2. **Redundancy (coding).** Send a few extra, cleverly computed bits so that the receiver can detect and correct errors. A good code at rate 1/2 reaches a BLER of $10^{-3}$ at roughly 2–3 dB, where uncoded BPSK would still make several percent bit errors.

The improvement is called **coding gain** and is measured in dB (Chapter 12). In practice coding gains of 5–8 dB are common — this is equivalent to multiplying the transmitter power by 3 to 6.

## 1.7 Simple real-life examples

- **Mobile phone call.** Your voice is coded at a few kbit/s, protected by channel codes and sent in short blocks every few milliseconds. In 5G the *control* part of this traffic (who transmits, when, with which settings) is protected by polar codes.
- **Deep-space probes.** Signals from Voyager arrive with power far below a billionth of a watt. Without very strong codes no picture of Neptune would exist.
- **QR codes and CDs.** A scratched CD still plays because of Reed–Solomon codes; a dirty QR code still scans for the same reason.
- **Wi-Fi and 4G/5G data.** Convolutional, turbo and LDPC codes protect the data channels.

**Professional insight**

A modern receiver does not send hard bits (0/1) to the decoder. It sends *soft* values that say how sure it is (the LLRs of Chapter 6). A value of $y = 0.05$ is a weak "0"; a value of $y = 1.8$ is a strong "0". Using this soft information gives about 2 dB extra gain over hard decisions — free gain that every good decoder, including all polar decoders in this book, uses.

## 1.8 Chapter summary

### Key concepts

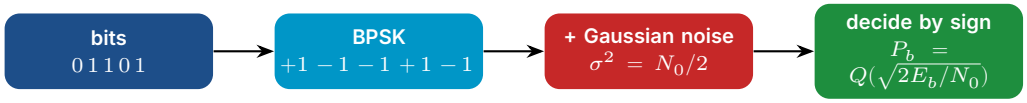
- A digital system sends bits; channel coding adds controlled redundancy against errors.
- Thermal noise is modelled as additive white Gaussian noise with variance $\sigma^2 = N_0/2$.
- BPSK maps $0 \to +1$, $1 \to -1$ and decides by the sign of the received value.

- BER counts wrong bits; BLER counts wrong blocks. Systems are specified in BLER.
- Buying reliability with power is expensive; coding is the efficient way.

## Important formulas

| | |
|---|---|
| AWGN | $y = s + w,\ w \sim \mathcal{N}(0, N_0/2)$ |
| BPSK BER | $P_b = Q(\sqrt{2E_b/N_0})$ |
| Decibels | $x_{\,\mathrm{dB}} = 10 \log_{10} x$ |
| Independent errors | $\mathrm{BLER} = 1 - (1-p)^K \approx Kp$ |

**Visual recap • Chapter 1**




One wrong bit ⇒ wrong block: $\mathrm{BLER} = 1 - (1-p)^K$.
$K = 1000,\ p = 10^{-3} \Rightarrow \mathrm{BLER} = 63\% \Rightarrow$ **we need coding**.

## Common student mistakes

**! Common mistake**

- Using $Q(\sqrt{E_b/N_0})$ instead of $Q(\sqrt{2E_b/N_0})$ for BPSK. The factor 2 comes from $\sigma^2 = N_0/2$.
- Forgetting to convert dB to a linear ratio before using a formula.
- Mixing $E_s/N_0$ (per transmitted symbol) and $E_b/N_0$ (per information bit). With coding they differ: $E_s/N_0 = R\,E_b/N_0$ (Chapter 2).
- Applying $1-(1-p)^K$ to decoded (coded) bits, whose errors are not independent.

## Practice problems

**Problem 1.1.** Uncoded BPSK operates at $E_b/N_0 = 8\,\mathrm{dB}$. Find the BER.

**Solution** $10^{0.8} = 6.31$, $\sqrt{12.62} = 3.55$, $P_b = Q(3.55) \approx 1.9 \times 10^{-4}$.

**Problem 1.2.** A link needs BLER $\leq 10^{-2}$ for $K = 200$-bit blocks with independent errors. What raw BER is needed?

**Solution** $1-(1-p)^{200} \leq 0.01 \Rightarrow p \leq 1 - 0.99^{1/200} = 5.03 \times 10^{-5}$ (the approximation $p \approx 0.01/200 = 5 \times 10^{-5}$ is almost the same).

**Problem 1.3.** In BPSK with $\sigma = 0.5$, what is the probability that the received value has the wrong sign?

**Solution** $P_b = Q(1/\sigma) = Q(2) = 0.0228$.

**Problem 1.4.** A receiver measures $E_b/N_0 = 3\,\text{dB}$. Express it as a ratio, and give $\sigma$ for uncoded BPSK with $E_s = 1$.

**Solution** $10^{0.3} = 1.995$. $N_0 = E_b/1.995 = 0.501$, $\sigma = \sqrt{N_0/2} = 0.5006$.

## Self-assessment quiz

1. True or false: the AWGN noise sample at time $t$ depends on the sample at time $t-1$.
2. Which block adds redundancy: the source encoder or the channel encoder?
3. By roughly how many dB must uncoded BPSK increase power to go from BER $10^{-3}$ to $10^{-5}$?
4. Why does a system designer care more about BLER than BER?

Answers: 1) False (white = independent). 2) Channel encoder. 3) About 2.8 dB. 4) A block with one wrong bit is useless or must be re-sent.

CHAPTER

# 2



# Fundamentals of Channel Coding

**IN THIS CHAPTER YOU WILL LEARN**

- what redundancy is and why it helps;
- codewords, block codes and the code rate $R = K/N$;
- Hamming distance and how many errors a code can fix;
- generator and parity-check matrices of linear codes;
- the three classic channels: BEC, BSC, AWGN;
- Shannon capacity and the channel coding theorem;
- why polar codes were a historic result.

## 2.1 What is channel coding?

**Intuition**

When you spell your name on a bad phone line you say "M as in Mary, A as in Alpha…". You add extra words that carry *no new information* but make the important letters easy to recover. This is redundancy. Channel coding is the mathematics of adding the *smallest* and *smartest* amount of redundancy that still lets the receiver fix the errors.

**Definition 2.1 • Channel coding**

**Channel coding** (also forward error correction, FEC) maps a block of $K$ information bits $\mathbf{m} = (m_0, \dots, m_{K-1})$ to a longer block of $N$ coded bits $\mathbf{c} = (c_0, \dots, c_{N-1})$, $N > K$. The $N - K$ extra bits are the **redundancy**. The receiver uses the known structure of the code to detect and correct transmission errors.

### 2.1.1 The simplest code: repetition

Send every bit three times: $0 \to 000$, $1 \to 111$. The decoder takes a *majority vote*.

**Worked Example 2.1 • Repetition code over a binary symmetric channel**

Each transmitted bit is flipped independently with probability $p = 0.1$. The majority vote fails only if two or three of the three copies are flipped:

$$P_e = \binom{3}{2} p^2(1-p) + p^3 = 3(0.01)(0.9) + 0.001 = 0.028.$$

The error probability dropped from 0.1 to 0.028 — but we now send three bits for every information bit. The *rate* is only 1/3. Good codes do much better than this: they reduce errors dramatically while keeping a high rate.

## 2.2 Codewords, block codes and code rate

**Definition 2.2 • Block code**

An $(N, K)$ **block code** $\mathcal{C}$ is a set of $2^K$ binary vectors of length $N$, called **codewords**. The encoder is a one-to-one map from the $2^K$ possible messages to the codewords.

- $K$ = number of **information bits** (message length);
- $N$ = number of **coded bits** (block length, codeword length);
- $R = K/N$ = **code rate**, the fraction of useful information in each coded bit.

$$R = \frac{K}{N}, \qquad E_s = R\,E_b \implies \frac{E_s}{N_0} = R\,\frac{E_b}{N_0}.$$

The second relation is very important for fair comparisons. Each coded symbol has energy $E_s$; it carries only $R$ information bits, so the energy spent per *information* bit is $E_b = E_s/R$. When we compare a coded system with an uncoded one at the same $E_b/N_0$, the coded system has *noisier* symbols but can correct errors. A code is good only if it wins this fair comparison.

**Worked Example 2.2 • Converting $E_b/N_0$ to noise variance**

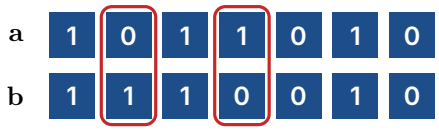
A rate-1/2 code with BPSK ($E_s = 1$) is simulated at $E_b/N_0 = 2\,\text{dB}$. Then $E_s/N_0 = 0.5 \times 10^{0.2} = 0.792$, so $N_0 = 1/0.792 = 1.262$ and $\sigma^2 = N_0/2 = 0.631$, $\sigma = 0.794$. Every simulation in this book uses exactly this conversion (function `awgn()` in Chapter 11).

## 2.3 Hamming distance

**Definition 2.3 • Hamming weight and distance**

The **Hamming weight** $w(\mathbf{a})$ is the number of ones in $\mathbf{a}$. The **Hamming distance** $d(\mathbf{a}, \mathbf{b})$ is the number of positions in which $\mathbf{a}$ and $\mathbf{b}$ differ, $d(\mathbf{a}, \mathbf{b}) = w(\mathbf{a} \oplus \mathbf{b})$. The **minimum distance** of a code is

$$d_{\min} = \min_{\mathbf{a} \neq \mathbf{b} \in \mathcal{C}} d(\mathbf{a}, \mathbf{b}).$$




$d(\mathbf{a}, \mathbf{b}) = 2$ (two positions differ)

**Intuition**

Think of codewords as cities on a map, and errors as steps away from a city. If any two cities are at least $d_{\min}$ steps apart, then after $t$ steps you are still closer to your own city as long as $2t < d_{\min}$. Larger distance between codewords means more errors can be absorbed.

$$\text{detect up to } d_{\min} - 1 \text{ errors}, \qquad \text{correct up to } t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor \text{ errors}.$$

For the repetition code $\{000, 111\}$, $d_{\min} = 3$, so it corrects $t = 1$ error — exactly what the majority vote does.

## 2.4 Linear block codes and generator matrices

Storing $2^K$ codewords in a table is impossible for $K = 100$ ($2^{100} \approx 10^{30}$). Linear codes need only a $K \times N$ matrix.

**Definition 2.4 • Linear block code and generator matrix**

A code is **linear** if the XOR (modulo-2 sum) of any two codewords is again a codeword. Every linear $(N, K)$ code has a $K \times N$ **generator matrix** $\mathbf{G}$ whose rows are a basis of the code, and encoding is a vector–matrix product over GF(2) (arithmetic modulo 2: $1 \oplus 1 = 0$):

$$\mathbf{c} = \mathbf{m}\,\mathbf{G}.$$

A **parity-check matrix** $\mathbf{H}$, of size $(N - K) \times N$, satisfies $\mathbf{H}\mathbf{c}^T = \mathbf{0}$ for every codeword.

For a linear code, $d(\mathbf{a}, \mathbf{b}) = w(\mathbf{a} \oplus \mathbf{b})$ and $\mathbf{a} \oplus \mathbf{b}$ is itself a codeword, so $d_{\min}$ equals the smallest weight of a non-zero codeword. This makes analysis much easier.

**Worked Example 2.3 • The (7,4) Hamming code**

Take the systematic generator matrix $\mathbf{G} = [\mathbf{I}_4 \,|\, \mathbf{P}]$:

$$\mathbf{G} = \left[\begin{array}{cccc|ccc} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{array}\right], \qquad \mathbf{H} = [\mathbf{P}^T \,|\, \mathbf{I}_3] = \left[\begin{array}{cccc|ccc} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{array}\right].$$

**Encoding** $\mathbf{m} = (1, 0, 1, 1)$: add rows 1, 3 and 4 of $\mathbf{G}$ (the positions of the ones in $\mathbf{m}$):

$$\begin{array}{r} 1\,0\,0\,0\,1\,1\,0 \\ \oplus\, 0\,0\,1\,0\,1\,1\,1 \\ \oplus\, 0\,0\,0\,1\,1\,0\,1 \\ \hline \mathbf{c} = 1\,0\,1\,1\,1\,0\,0 \end{array}$$

**Checking**: $\mathbf{H}\mathbf{c}^T = (0, 0, 0)^T$. **An error** in the third bit gives $\mathbf{r} = 1001100$ and the *syndrome* $\mathbf{H}\mathbf{r}^T = (1, 1, 1)^T$, which is exactly the third column of $\mathbf{H}$: the decoder knows where the error

is and flips it back. The weights of the 15 non-zero codewords are 3 (seven times), 4 (seven times) and 7 (once), so $d_{\min} = 3$ and one error is corrected.

**Important note**

Polar codes are linear block codes. Their generator matrix is built in a very special way (Chapter 5) that makes both encoding and decoding fast. Everything in this section — rate, distance, generator matrix — applies to them directly.

## 2.5 Three channel models

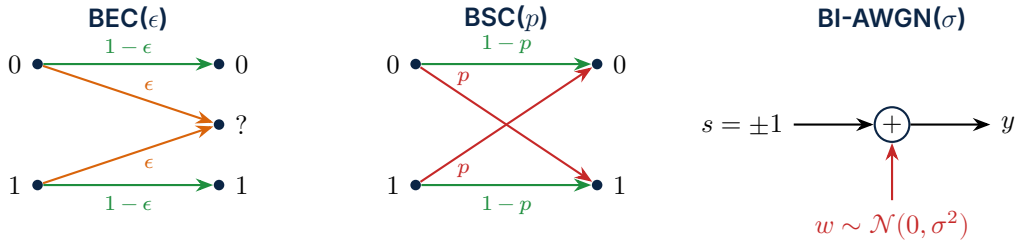



**Figure 2.1.** The binary erasure channel, the binary symmetric channel and the binary-input AWGN channel.

- **Binary erasure channel BEC($\epsilon$).** Each bit is either received correctly or *erased* (the receiver knows it is lost, output "?") with probability $\epsilon$. Unrealistic but perfect for teaching: polar codes are easiest to analyse on the BEC (Chapter 4).
- **Binary symmetric channel BSC($p$).** Each bit is flipped with probability $p$ and the receiver does not know which bits were flipped. This is a hard-decision channel.
- **Binary-input AWGN channel.** BPSK plus Gaussian noise, the model of Chapter 1. The receiver has soft information. This is the main channel for practical design.

## 2.6 Shannon channel capacity

**Intuition**

Every channel has a "speed limit". Below that limit you can make errors as rare as you wish with long enough codes; above it errors are unavoidable whatever you do. Shannon proved this in 1948 and called the limit the *capacity* $C$, measured in bits per channel use.

**Definition 2.5 • Capacity**

For a channel with input $X$ and output $Y$, the **capacity** is the largest mutual information over all input distributions, $C = \max_{p(x)} I(X;Y)$. For symmetric binary channels the maximum is reached with equally likely inputs, and the **symmetric capacity** is denoted $I(W)$.

$$C_{\mathrm{BEC}} = 1-\epsilon, \qquad C_{\mathrm{BSC}} = 1 - H_2(p), \qquad C_{\mathrm{AWGN}} = \tfrac{1}{2}\log_2\!\left(1+\tfrac{2E_s}{N_0}\right),$$

where $H_2(p) = -p\log_2 p - (1-p)\log_2(1-p)$ is the binary entropy function.

The AWGN formula is for a real channel with Gaussian inputs; with BPSK inputs the capacity is lower and must be computed numerically (Fig. 2.2).

**Derivation of $C_{\mathrm{BEC}}$.** With equally likely inputs, the output is "?" with probability $\epsilon$ and then tells us nothing; otherwise it tells us the bit exactly (1 bit). So on average $I(X;Y) = (1-\epsilon)\cdot 1 + \epsilon\cdot 0 = 1-\epsilon$. Intuitively, a fraction $\epsilon$ of the bits is lost, so at most a fraction $1-\epsilon$ can carry information. Remarkably, this is achievable even though the transmitter does not know *which* bits will be erased.

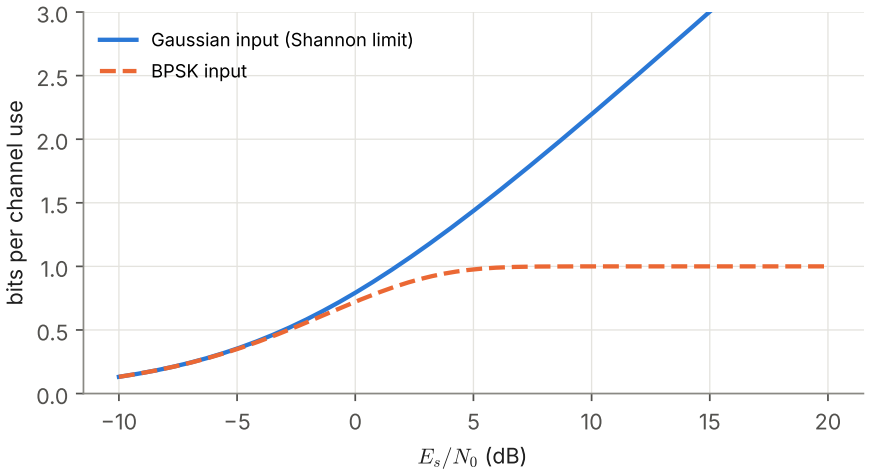



**Figure 2.2.** Capacity of the real AWGN channel: Gaussian inputs versus BPSK inputs.

**Definition 2.6 • Channel coding theorem (Shannon, 1948)**

For every rate $R < C$ there exist codes of rate $R$ whose block error probability tends to zero as $N \to \infty$. For every rate $R > C$ the error probability is bounded away from zero for all codes.

### 2.6.1 The Shannon limit in $E_b/N_0$

Setting $R = C$ with $E_s = RE_b$ in the AWGN formula gives the smallest usable $E_b/N_0$:

$$R = \tfrac{1}{2}\log_2\!\left(1+2R\frac{E_b}{N_0}\right) \implies \left.\frac{E_b}{N_0}\right|_{\min} = \frac{2^{2R}-1}{2R}.$$

**Worked Example 2.4 • Shannon limits**

| Rate $R$ | 1/4 | 1/2 | 3/4 |
|---|---|---|---|
| Gaussian input: $(2^{2R}-1)/(2R)$ | −0.82 dB | 0.00 dB | 0.86 dB |
| BPSK input (numerical) | −0.79 dB | 0.19 dB | 1.63 dB |

For $R \to 0$ the Gaussian limit becomes $\ln 2 = -1.59$ dB, the famous *ultimate Shannon limit*. A practical rate-1/2 code that reaches BLER $10^{-3}$ at 2 dB is therefore about 1.8 dB from the BPSK limit. The size of this gap is the main figure of merit of a code family.

## 2.7 Why polar codes are important

Shannon's proof was not constructive: it used *random* codes, which nobody can decode. For 60 years researchers built codes that came close to capacity in practice (turbo codes in 1993, LDPC codes rediscovered in the 1990s), but none was *proved* to reach capacity with a practical decoder on a general channel.

**Why the 2009 result was historic**

Polar codes, introduced by Erdal Arıkan in 2009, were the first family of codes with *all* of these properties together:

1. an explicit, deterministic construction (no randomness);
2. proved to achieve the symmetric capacity of any binary-input memoryless symmetric channel;
3. encoding *and* decoding with complexity $O(N \log N)$.

In 2016 3GPP selected polar codes for the control channels of 5G New Radio. They are now in billions of devices.

## 2.8 Chapter summary

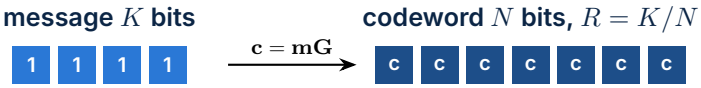
### Key concepts

- An $(N, K)$ code maps $K$ information bits to $N$ coded bits; the rate is $R = K/N$.
- Fair comparisons use $E_b/N_0$; the coded symbol SNR is $E_s/N_0 = R\,E_b/N_0$.
- Minimum distance $d_{\min}$ decides how many errors can be detected $(d_{\min} - 1)$ and corrected $(\lfloor (d_{\min} - 1)/2 \rfloor)$.
- Linear codes are described by a generator matrix $\mathbf{G}$ $(\mathbf{c} = \mathbf{mG})$ and a parity-check matrix $\mathbf{H}$.
- Capacity is the maximum reliable rate; polar codes provably achieve it with $O(N \log N)$ complexity.

### Important formulas

| | |
|---|---|
| Rate and energy | $R = K/N, \quad E_s/N_0 = R\,E_b/N_0$ |
| Error correction | $t = \lfloor (d_{\min} - 1)/2 \rfloor$ |
| Capacities | $1 - \epsilon$ (BEC), $1 - H_2(p)$ (BSC), $\frac{1}{2}\log_2(1 + 2E_s/N_0)$ (AWGN) |
| Shannon limit | $E_b/N_0 \geq (2^{2R} - 1)/(2R)$ |

**Visual recap • Chapter 2**




distance between codewords ⇒ error correction • capacity $C$ = speed limit • polar codes: provably reach $C$ with $O(N \log N)$

## Common student mistakes

**! Common mistake**

- Comparing a coded and an uncoded system at the same $E_s/N_0$ — this hides the energy cost of redundancy.
- Thinking capacity is a property of the code. It is a property of the *channel*; codes try to reach it.
- Using ordinary addition instead of XOR in $\mathbf{mG}$.
- Believing a code can correct $d_{\min}$ errors. It corrects $\lfloor (d_{\min}-1)/2 \rfloor$ with a bounded-distance decoder.

## Practice problems

**Problem 2.1.** A code has $N = 256$ and $K = 64$. What is its rate? At $E_b/N_0 = 1$ dB, what is $\sigma$ for BPSK with $E_s = 1$?

**Solution** $R = 0.25$. $E_s/N_0 = 0.25 \times 10^{0.1} = 0.3147$, $N_0 = 3.177$, $\sigma = \sqrt{N_0/2} = 1.260$.

**Problem 2.2.** Find the capacity of a BSC with $p = 0.11$ and of a BEC with $\epsilon = 0.5$.

**Solution** $H_2(0.11) = 0.4999$, so $C_{\text{BSC}} \approx 0.500$; $C_{\text{BEC}} = 0.5$. Both channels can carry half a bit per use.

**Problem 2.3.** A linear code has $d_{\min} = 6$. How many errors can it detect and correct?

**Solution** Detect 5; correct $\lfloor 5/2 \rfloor = 2$.

**Problem 2.4.** Encode $\mathbf{m} = (0, 1, 1, 0)$ with the (7,4) Hamming code of this chapter.

**Solution** Rows 2 and 3: $0100011 \oplus 0010111 = 0110100$. Check: $\mathbf{Hc}^T = \mathbf{0}$.

**Problem 2.5.** What is the minimum $E_b/N_0$ for reliable communication at $R = 1/3$ with Gaussian inputs?

**Solution** $(2^{2/3} - 1)/(2/3) = 0.881 \Rightarrow -0.55$ dB.

## Self-assessment quiz

1. Is the set $\{000, 011, 101, 110\}$ a linear code? What is $d_{\min}$?

2. What happens to $E_s/N_0$ if we halve the code rate at fixed $E_b/N_0$?
3. What is the capacity of a BEC with $\epsilon = 0.2$?
4. Name the three properties that made polar codes historic.

Answers: 1) Yes (closed under XOR), $d_{\min} = 2$. 2) It drops by 3 dB. 3) 0.8. 4) Explicit construction, provably capacity-achieving, $O(N \log N)$ encoding and decoding.

PART II

# Intermediate Level: Understanding Polar Codes

Channel polarization, the Kronecker generator matrix and the butterfly encoder, step by step with real numbers.

CHAPTER

# 3



# Introduction to Polar Codes

**IN THIS CHAPTER YOU WILL LEARN**

- what a polar code is, in one picture;
- the history of Arıkan's 2009 discovery;
- channel polarization explained without equations;
- reliable and unreliable bit-channels;
- frozen bits and information bits;
- the parameters $N$, $K$, $R$ and the sets $\mathcal{A}$, $\mathcal{F}$;
- how polar codes compare with LDPC and turbo codes.

## 3.1 What are polar codes?

**The classroom analogy**

A teacher has $N$ seats in a classroom. Some seats are near the board and have a perfect view; others are behind a pillar and see nothing. If the teacher must place $K$ important students, she puts them in the $K$ best seats and leaves the pillar seats with dummy "students" whose answers she already knows. A polar code does exactly this with bits: it creates $N$ "seats" (bit-channels) of very different quality, puts the $K$ information bits in the best ones, and fills the bad ones with known values (zeros) called *frozen bits*.

**Definition 3.1 • Polar code**

A **polar code** of length $N = 2^n$ and dimension $K$ is a linear block code defined by

1. the generator matrix $\mathbf{G}_N = \mathbf{F}^{\otimes n}$ with kernel $\mathbf{F} = \left[\begin{smallmatrix} 1 & 0 \\ 1 & 1 \end{smallmatrix}\right]$;
2. an **information set** $\mathcal{A} \subset \{0, \dots, N-1\}$ with $|\mathcal{A}| = K$; the complement $\mathcal{F} = \{0, \dots, N-1\} \setminus \mathcal{A}$ is the **frozen set**.

The encoder builds a vector $\mathbf{u}$ of length $N$ with $u_i =$ message bits for $i \in \mathcal{A}$ and $u_i = 0$ for $i \in \mathcal{F}$, and sends $\mathbf{x} = \mathbf{u}\,\mathbf{G}_N$.

That is all! The magic is entirely in *which* positions are frozen and in the structure of $\mathbf{G}_N$, which makes the bit positions behave like channels of very different quality.

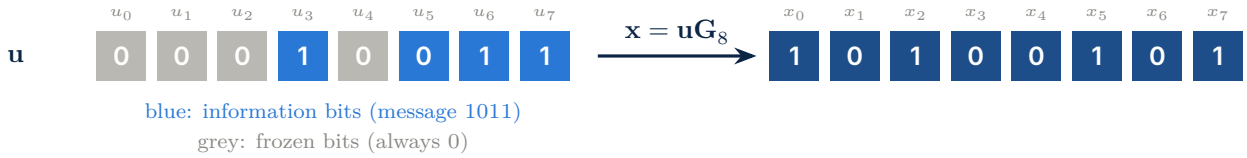
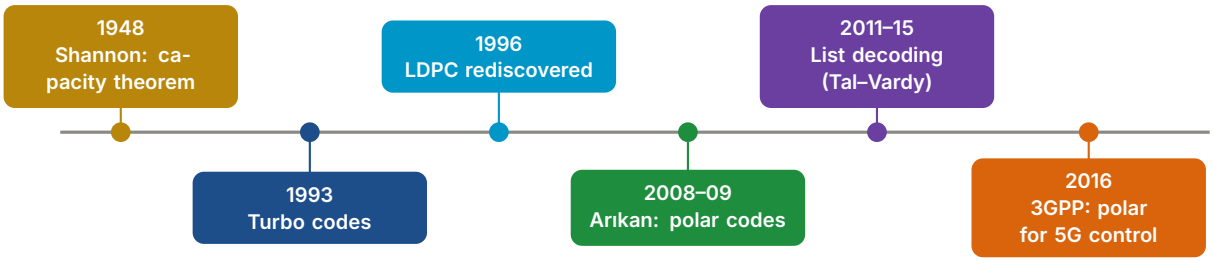



**Figure 3.1.** A complete $(8, 4)$ polar code: frozen set $\mathcal{F} = \{0, 1, 2, 4\}$, information set $\mathcal{A} = \{3, 5, 6, 7\}$. This code is used in many examples of this book.

## 3.2 History: Arıkan's 2009 breakthrough




- Erdal Arıkan (Bilkent University, Ankara) had worked for many years on sequential decoding and the "cutoff rate" of channels. While trying to boost the cutoff rate he noticed that combining two copies of a channel and splitting them again creates one better and one worse channel. Repeating the operation recursively, the channels *polarize*.
- His paper "Channel polarization: a method for constructing capacity-achieving codes for symmetric binary-input memoryless channels" appeared in the *IEEE Transactions on Information Theory* in July 2009 and received the IEEE Information Theory Society Paper Award.
- Early polar codes with the plain SC decoder were weaker than LDPC codes at practical lengths. The list decoder of Tal and Vardy (conference version 2011, journal 2015) combined with a CRC changed this completely: at short and medium lengths CRC-aided list decoding became competitive with or better than the best known codes.
- In November 2016 (3GPP RAN1 meeting #87) polar codes were agreed for the control channels of the 5G enhanced mobile broadband scenario. They are specified in 3GPP TS 38.212.

## 3.3 Why were polar codes invented?

Before 2009 the situation was strange: Shannon had proved that capacity can be reached, practical codes came close in simulations, but *no* code was proved to reach capacity with a practical decoder on general binary channels. Polar codes closed this gap with a beautifully simple idea: do not try to correct errors on a uniform channel; instead *transform* $N$ identical channels into channels that are either almost perfect or almost useless, and then the coding problem becomes trivial — send data on the perfect ones.

## 3.4 Channel polarization explained intuitively

**Two weak friends**

Two friends each pass you a coin through a foggy window (each sees the coin correctly with probability 0.9). You ask the first friend to tell you $u_0 \oplus u_1$ and the second to tell you $u_1$. To learn $u_0$ you need *both* messages to be right — harder than before. But once you know $u_0$, you get *two* independent opinions about $u_1$ — easier than before. From two equal channels you have made one *worse* and one *better* channel. The total information is unchanged; it has only been re-distributed.

Repeat the trick: combine two "better" channels to make an even better one and a slightly worse one; combine two "worse" ones to make an even worse one and a slightly better one. After $n$ levels there are $N = 2^n$ synthetic channels, called **bit-channels**, and almost all of them are extreme.

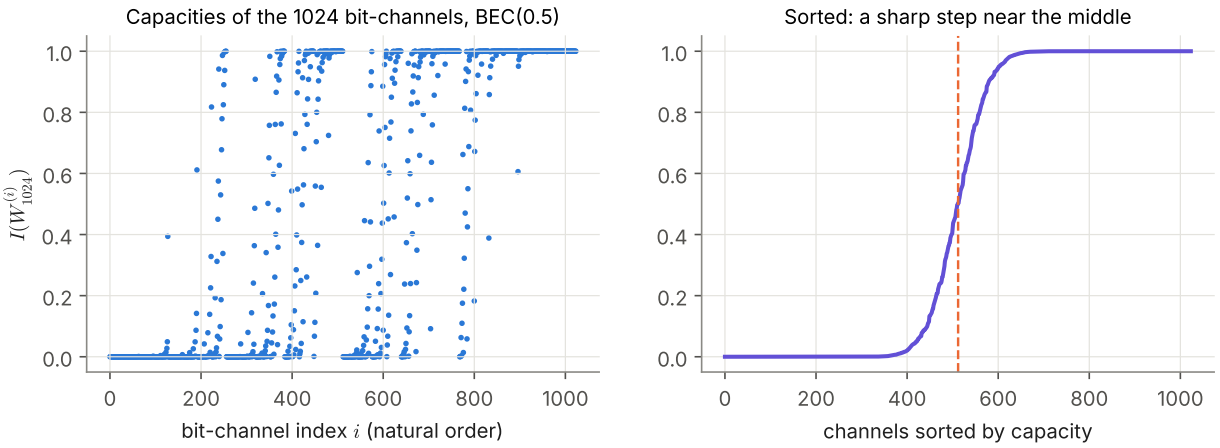



**Figure 3.2.** Polarization over BEC(0.5) with $N = 1024$. Left: capacity of each bit-channel in natural order. Right: sorted — about half the channels are almost perfect and half almost useless.

## 3.5 Reliable and unreliable bit-channels

**Definition 3.2 • Bit-channel**

The $i$-th **bit-channel** $W_N^{(i)}$ is the channel that bit $u_i$ "sees" when the receiver knows the channel output $\mathbf{y}$ *and* the correct values of all earlier bits $u_0, \dots, u_{i-1}$. Its quality is measured by its capacity $I(W_N^{(i)}) \in [0,1]$ or by its Bhattacharyya parameter $Z(W_N^{(i)}) \in [0,1]$ (Chapter 4): small $Z$ means reliable.

The "knows the earlier bits" part is essential. It is exactly what a *successive* decoder can offer: it decodes $u_0$ first, then $u_1$ using $\hat{u}_0$, then $u_2$ using $\hat{u}_0, \hat{u}_1$, and so on (Chapter 6).

**Definition 3.3 • Polarization theorem (Arıkan 2009)**

For any binary-input memoryless symmetric channel $W$ and any $\delta \in (0,1)$, as $N \to \infty$,

$$\frac{\left|\{i : I(W_N^{(i)}) > 1-\delta\}\right|}{N} \to I(W), \qquad \frac{\left|\{i : I(W_N^{(i)}) < \delta\}\right|}{N} \to 1 - I(W).$$

In words: the fraction of almost-perfect channels tends to the capacity of the original channel, and all the other channels become almost useless.

## 3.6 Frozen bits and information bits

**Definition 3.4 • Frozen and information bits**

**Information bits** are placed on the $K$ most reliable bit-channels (the set $\mathcal{A}$). **Frozen bits** are placed on the remaining $N-K$ least reliable channels (the set $\mathcal{F}$); their values are fixed in advance and known to the decoder. With symmetric channels we can take them all equal to 0.

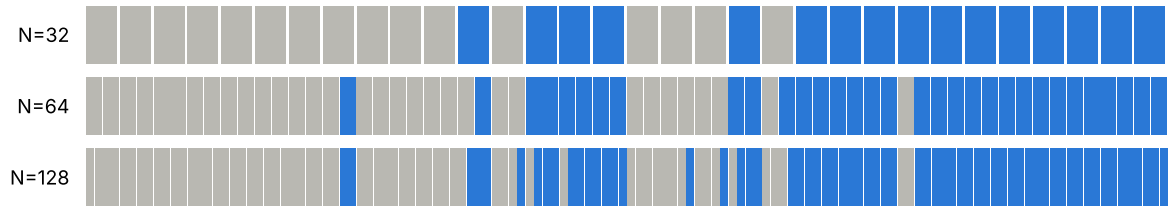



**Figure 3.3.** Information (blue) and frozen (grey) positions of rate-1/2 polar codes designed by Gaussian approximation. Frozen positions cluster at small indices, information at large indices, with a mixed zone in the middle.

**Frozen bits are not wasted**

The frozen positions do not carry data, but they are not useless: knowing them for certain is what allows the decoder to use them as "anchors". A bit that would arrive almost random is replaced by a bit the decoder knows perfectly. Choosing which positions to freeze is called **code construction** and is the heart of polar code design (Chapters 4 and 9).

## 3.7 Block length, information length and code rate

| | |
|---|---|
| $N = 2^n$ | block length; always a power of two for the basic construction ($n = \log_2 N$ levels). |
| $K$ | number of information bits (including CRC bits when a CRC is used). |
| $R = K/N$ | code rate. |
| $\mathcal{A}$, $\mathcal{F}$ | information set ($K$ indices), frozen set ($N-K$ indices). |

Lengths that are not powers of two are obtained by puncturing, shortening or repetition (Chapter 9); 5G uses this to reach any number of transmitted bits.

## 3.8 Capacity-achieving behaviour

**Intuition**

If we choose $K = \lfloor NR \rfloor$ with $R < I(W)$, the polarization theorem says that for large $N$ there are more than $K$ almost-perfect channels. All information bits sit on almost-perfect channels, so all of them are decoded correctly with high probability. The rate $R$ can be as close to $I(W)$ as we like.

Arıkan proved the bound

$$P_e^{\mathrm{SC}}(N, K, \mathcal{A}) \;\le\; \sum_{i \in \mathcal{A}} Z(W_N^{(i)}),$$

and Arıkan and Telatar showed that, for any $\beta < 1/2$, the sum can be made smaller than $2^{-N^\beta}$ at any fixed rate below capacity. So the block error probability falls roughly like $2^{-\sqrt{N}}$ — fast, but slower than the exponential $2^{-cN}$ of the best (non-practical) random codes. This is one reason why plain polar codes need help (list decoding and CRC) at short lengths.

## 3.9 Polar codes versus LDPC and turbo codes

| | **Turbo codes** | **LDPC codes** | **Polar codes** |
|---|---|---|---|
| **Invented** | Berrou et al., 1993 | Gallager 1962; rediscovered 1990s | Arıkan 2009 |
| **Structure** | two convolutional codes + interleaver | sparse parity-check matrix | Kronecker power $\mathbf{F}^{\otimes n}$ + frozen set |
| **Typical decoder** | iterative BCJR | belief propagation (iterative) | SC, SCL, CA-SCL (sequential) |
| **Capacity proof** | no | for special ensembles/channels (e.g. spatially coupled) | yes, for all BMS channels with SC |
| **Strength** | mature, flexible | very high throughput, long blocks | excellent at short lengths with CA-SCL; simple rate adaption |
| **Weakness** | error floors, latency | short-block performance, design of irregular graphs | sequential decoding limits throughput |
| **Used in** | 3G, 4G LTE data | 5G NR data, Wi-Fi, DVB-S2 | 5G NR control and broadcast channels |

**Professional insight**

The 5G decision was not "polar is better than LDPC". It was a split by job: LDPC for large, high-speed data blocks, polar for small, critical control messages (typically tens to a few hundred bits) where very low BLER and good short-length performance matter most.

## 3.10 Chapter summary

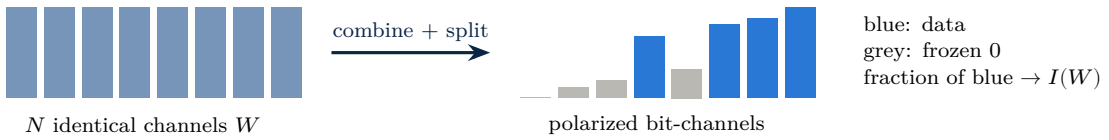
### Key concepts

- A polar code is $\mathbf{x} = \mathbf{u}\mathbf{G}_N$ with $\mathbf{G}_N = \mathbf{F}^{\otimes n}$ and a chosen frozen set.
- Recursive channel combining and splitting make bit-channels polarize into almost-perfect and almost-useless.
- Information bits go on the $K$ most reliable bit-channels; the others are frozen to 0.
- The fraction of good channels tends to $I(W)$: polar codes achieve capacity.
- 5G NR uses polar codes for control information and LDPC codes for data.

### Important formulas

| | |
|---|---|
| Encoding | $\mathbf{x} = \mathbf{u}\,\mathbf{F}^{\otimes n}$, $u_i = 0$ for $i \in \mathcal{F}$ |
| Rate | $R = K/N = \lvert\mathcal{A}\rvert/N$ |
| SC error bound | $P_e \le \sum_{i\in\mathcal{A}} Z(W_N^{(i)})$ |

**Visual recap • Chapter 3**




### Common student mistakes

**! Common mistake**

- Thinking that frozen positions are the *first* $N-K$ indices. They are the *least reliable* indices, which are mostly but not only small (see Fig. 3.3).
- Forgetting that the bit-channel definition assumes the earlier bits are known. The decoder can only approximate this by using its own earlier decisions.
- Believing polar codes are good at every length with the SC decoder. At short lengths SC is weak; list decoding with CRC is needed.

### Practice problems

**Problem 3.1.** A polar code has $N = 512$ and rate $R = 3/8$. How many frozen bits does it have?

**Solution** $K = 192$, so $N - K = 320$ frozen bits.

**Problem 3.2.** A BEC has $\epsilon = 0.3$. For very large $N$, roughly what fraction of bit-channels is almost perfect?

**Solution** $I(W) = 1 - \epsilon = 0.7$: about 70% of the bit-channels.

**Problem 3.3.** The Bhattacharyya parameters of the information bit-channels of a code are 0.004, 0.12, 0.19 and 0.32. Give an upper bound on the SC block error probability.

**Solution** $P_e \leq 0.004 + 0.12 + 0.19 + 0.32 = 0.634$. (This is the $(8, 4)$ code over BEC(0.5); the bound is loose but illustrates the method.)

## Self-assessment quiz

1. What is the kernel matrix of Arıkan's polar codes?
2. Which bits does the decoder already know before decoding?
3. In which 5G channels are polar codes used?
4. What happens to the fraction of "middle" (not polarized) channels as $N$ grows?

Answers: 1) $\mathbf{F} = [1\,0; 1\,1]$. 2) The frozen bits. 3) Control and broadcast channels (DCI, UCI, PBCH). 4) It tends to zero.

CHAPTER

# 4



# Channel Polarization — The Core Idea

**IN THIS CHAPTER YOU WILL LEARN**

- channel combining: building $W_2$, $W_4$, …, $W_N$;
- channel splitting: the synthetic bit-channels $W_N^{(i)}$;
- the exact BEC recursion $Z \mapsto 2Z - Z^2$, $Z \mapsto Z^2$;
- the Bhattacharyya parameter and its meaning;
- how to compute the reliability of every bit-channel;
- why channels polarize when $N$ grows.

## 4.1 Channel combining

**Intuition**

We start with two independent uses of the same channel $W$. Instead of sending two independent bits, we *mix* them: the first channel carries $u_0 \oplus u_1$, the second carries $u_1$. Nothing is lost — from $(x_0, x_1)$ we can always recover $(u_0, u_1)$ — but the two bits are now protected in different ways.

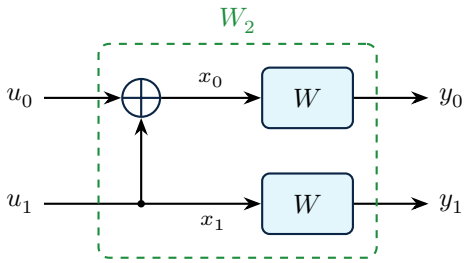



**Figure 4.1.** The basic combining step: two copies of $W$ become the vector channel $W_2 : (u_0, u_1) \mapsto (y_0, y_1)$.

**Definition 4.1 • Combined channel $W_2$**

With $x_0 = u_0 \oplus u_1$ and $x_1 = u_1$, i.e. $\mathbf{x} = \mathbf{uF}$, the combined channel has transition probability

$$W_2(y_0, y_1 \mid u_0, u_1) = W(y_0 \mid u_0 \oplus u_1)\, W(y_1 \mid u_1).$$

### 4.1.1 Recursive combining

The construction is applied again to two copies of $W_2$. In general two independent copies of $W_{N/2}$ are combined into $W_N$:

$$W_N(\mathbf{y} \mid \mathbf{u}) = W^N(\mathbf{y} \mid \mathbf{uG}_N) = \prod_{j=0}^{N-1} W(y_j \mid x_j), \qquad \mathbf{x} = \mathbf{uG}_N.$$

Fig. 4.2 shows $W_4$. With our convention $\mathbf{G}_N = \mathbf{F} \otimes \mathbf{G}_{N/2}$, the *second* half of $\mathbf{u}$ is XORed into the first half, and each half is then encoded by $\mathbf{G}_{N/2}$:

$$\mathbf{x} = \big[(\mathbf{u}_{\text{first}} \oplus \mathbf{u}_{\text{second}})\mathbf{G}_{N/2} \;\big|\; \mathbf{u}_{\text{second}}\mathbf{G}_{N/2}\big].$$

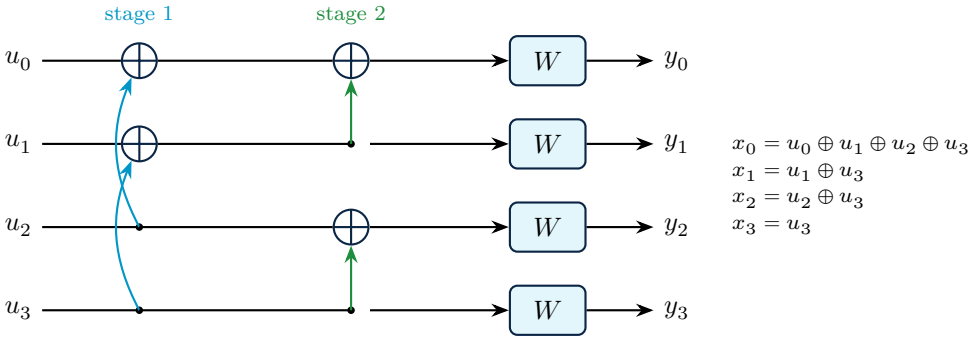



**Figure 4.2.** $W_4$ built from four copies of $W$. Each XOR node combines two signals; dots are copy points.

## 4.2 Channel splitting

Combining makes one big channel. *Splitting* cuts it again into $N$ binary-input channels, one per bit $u_i$, in the order in which a successive decoder will meet them.

**Definition 4.2 • Synthetic bit-channels**

The $i$-th bit-channel has input $u_i$ and output $(\mathbf{y}, u_0^{i-1})$ — the channel output plus all earlier bits:

$$W_N^{(i)}(\mathbf{y}, u_0^{i-1} \mid u_i) = \sum_{u_{i+1}^{N-1}} \frac{1}{2^{N-1}}\, W_N(\mathbf{y} \mid \mathbf{u}).$$

Later bits $u_{i+1}, \dots, u_{N-1}$ are unknown, so they are averaged out (treated as random).

For $N = 2$ we obtain the two most important channels in this book:

$$W^-(y_0, y_1 \mid u_0) = W_2^{(0)} = \tfrac{1}{2} \sum_{u_1 \in \{0,1\}} W(y_0 \mid u_0 \oplus u_1)\, W(y_1 \mid u_1) \qquad \text{(“minus”, worse)}$$

$$W^+(y_0, y_1, u_0 \mid u_1) = W_2^{(1)} = \tfrac{1}{2}\, W(y_0 \mid u_0 \oplus u_1)\, W(y_1 \mid u_1) \qquad \text{(“plus”, better)}$$

**Intuition**

To decide $u_0$ you must look through *both* channels at a mixture that also contains the unknown $u_1$: two noisy views multiplied together — worse. To decide $u_1$ you already know $u_0$, so $y_0$ tells you $u_1$ (after removing $u_0$) and $y_1$ tells you $u_1$ directly: two independent views — better. This is exactly the repetition code effect from Chapter 2.

### 4.2.1 Conservation and polarization of information

By the chain rule of mutual information, nothing is created or destroyed:

$$I(W^-) + I(W^+) = 2\,I(W), \qquad I(W^-) \le I(W) \le I(W^+),$$

with equality only when $I(W) \in \{0, 1\}$. *Derivation sketch.* $(u_0, u_1) \leftrightarrow (x_0, x_1)$ is one-to-one, so $I(U_0U_1; Y_0Y_1) = I(X_0X_1; Y_0Y_1) = 2I(W)$. The chain rule gives $I(U_0U_1; Y_0Y_1) = I(U_0; Y_0Y_1) + I(U_1; Y_0Y_1 \mid U_0)$, and since $U_0, U_1$ are independent, $I(U_1; Y_0Y_1 \mid U_0) = I(U_1; Y_0Y_1U_0) = I(W^+)$. The first term is $I(W^-)$.

## 4.3 The binary erasure channel: an exact example

On the BEC every synthetic channel is again a BEC, and the recursion is exact and simple.

**Worked Example 4.1 • Minus and plus channels of a BEC($\epsilon$)**

**Minus channel (bit $u_0$).** We need $x_0 = u_0 \oplus u_1$ and $x_1 = u_1$; then $u_0 = x_0 \oplus x_1$. We learn $u_0$ only if *both* $y_0$ and $y_1$ are not erased:

$$P(\text{not erased}) = (1-\epsilon)^2 \;\Rightarrow\; \epsilon^- = 1 - (1-\epsilon)^2 = 2\epsilon - \epsilon^2.$$

**Plus channel (bit $u_1$, with $u_0$ known).** $u_1 = x_1$ from $y_1$, or $u_1 = x_0 \oplus u_0$ from $y_0$. It is lost only if *both* are erased:

$$\epsilon^+ = \epsilon^2.$$

**Numbers for $\epsilon = 0.5$.** $\epsilon^- = 0.75$, $\epsilon^+ = 0.25$. Capacities: $I(W^-) = 0.25$, $I(W^+) = 0.75$, and indeed $0.25 + 0.75 = 2 \times 0.5$.

$$\text{BEC:} \qquad Z(W^-) = 2Z(W) - Z(W)^2, \qquad Z(W^+) = Z(W)^2, \qquad Z = \epsilon.$$

### 4.3.1 Going to $N = 4$ and $N = 8$

Apply the same rules to $W^-$ and $W^+$ (each with $\epsilon = 0.5$ start):

| $i$ | binary | path | erasure prob. $Z(W_4^{(i)})$ | capacity | rank |
|---|---|---|---|---|---|
| 0 | 00 | $-,-$ | $2(0.75) - 0.75^2 = 0.9375$ | 0.0625 | 4 (worst) |
| 1 | 01 | $-,+$ | $0.75^2 = 0.5625$ | 0.4375 | 3 |
| 2 | 10 | $+,-$ | $2(0.25) - 0.25^2 = 0.4375$ | 0.5625 | 2 |
| 3 | 11 | $+,+$ | $0.25^2 = 0.0625$ | 0.9375 | 1 (best) |

**Reading the index in binary**

With our natural-order convention, write $i$ in binary with $n$ bits, *most significant bit first*. Each bit tells which transform to apply, starting from the physical channel: $0 \Rightarrow$ minus ($Z \to 2Z - Z^2$), $1 \Rightarrow$ plus ($Z \to Z^2$). Example, $N = 8$, $i = 6 = 110_2$: $0.5 \xrightarrow{+} 0.25 \xrightarrow{+} 0.0625 \xrightarrow{-} 2(0.0625) - 0.0625^2 = 0.1211$.

| $i$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| binary | 000 | 001 | 010 | 011 | 100 | 101 | 110 | 111 |
| $Z(W_8^{(i)})$ | 0.9961 | 0.8789 | 0.8086 | 0.3164 | 0.6836 | 0.1914 | 0.1211 | 0.0039 |
| $I(W_8^{(i)})$ | 0.0039 | 0.1211 | 0.1914 | 0.6836 | 0.3164 | 0.8086 | 0.8789 | 0.9961 |

The four best channels (shaded) are $\{3, 5, 6, 7\}$ — the information set of the $(8, 4)$ code of Fig. 3.1. Notice that channel 4 is *worse* than channel 3 although its index is larger.

## 4.4 Visualizing polarization for N = 2, 4, 8, 16

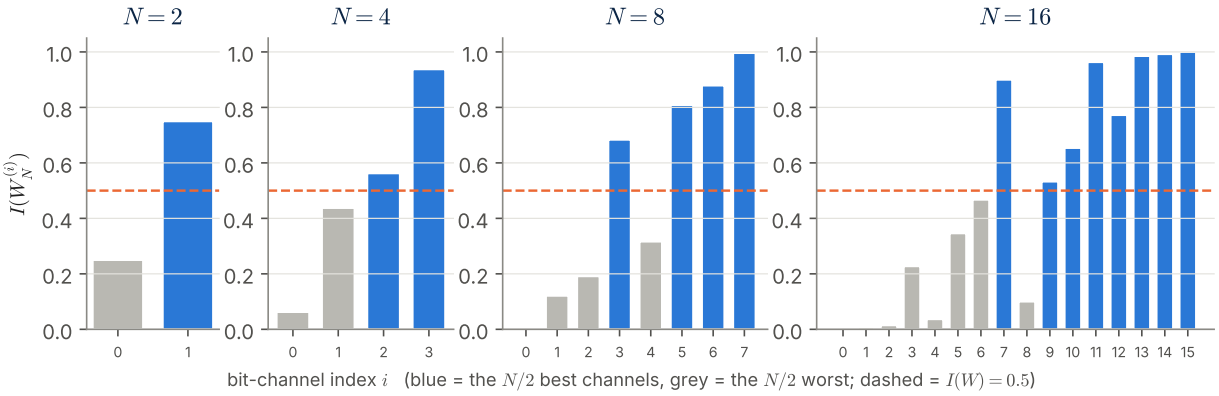



**Figure 4.3.** Capacities of the bit-channels for BEC(0.5). Already at $N = 16$ most bars are close to 0 or 1.

| $i$ | binary | $Z$ | $I$ | rank | $i$ | binary | $Z$ | $I$ | rank |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0000 | 0.99998 | 0.00002 | 16 | 8 | 1000 | 0.89989 | 0.10011 | 12 |
| 1 | 0001 | 0.99220 | 0.00780 | 15 | 9 | 1001 | 0.46730 | 0.53270 | 8 |
| 2 | 0010 | 0.98534 | 0.01466 | 14 | 10 | 1010 | 0.34618 | 0.65382 | 7 |
| 3 | 0011 | 0.77248 | 0.22752 | 11 | 11 | 1011 | 0.03664 | 0.96336 | 4 |
| 4 | 0100 | 0.96336 | 0.03664 | 13 | 12 | 1100 | 0.22752 | 0.77248 | 6 |
| 5 | 0101 | 0.65382 | 0.34618 | 10 | 13 | 1101 | 0.01466 | 0.98534 | 3 |
| 6 | 0110 | 0.53270 | 0.46730 | 9 | 14 | 1110 | 0.00780 | 0.99220 | 2 |
| 7 | 0111 | 0.10011 | 0.89989 | 5 | 15 | 1111 | 0.00002 | 0.99998 | 1 |

For BEC(0.5) there is a nice symmetry: $Z(W_{16}^{(i)}) + Z(W_{16}^{(15-i)}) = 1$. The best eight channels are $\{7, 9, 10, 11, 12, 13, 14, 15\}$; their $Z$ values add up to 1.2002 — still a poor SC bound at this tiny length.

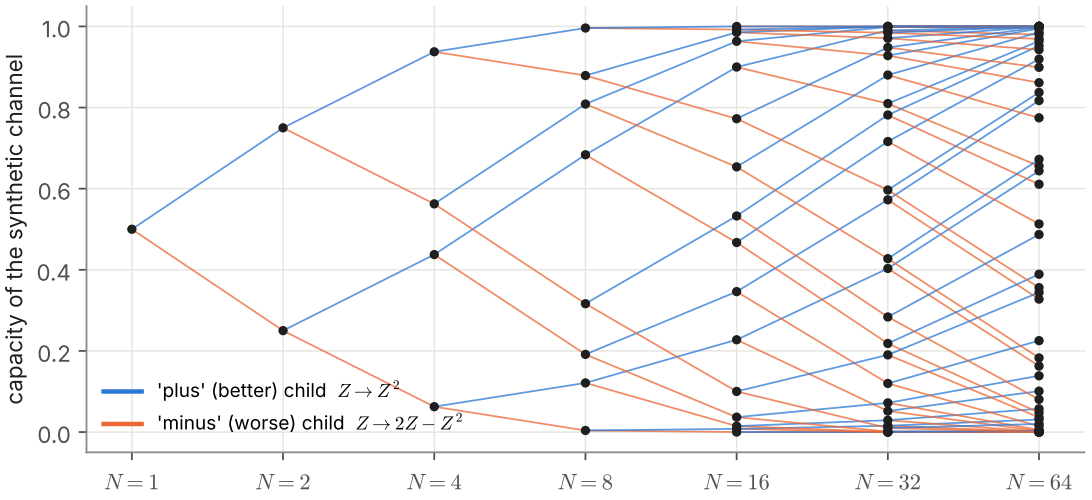



**Figure 4.4.** The polarization tree. Every channel splits into a better (blue) and a worse (orange) child. The paths spread towards 0 and 1.

## 4.5 The Bhattacharyya parameter

For general channels (BSC, AWGN) the synthetic channels are no longer of the same family, and we need a single number that measures reliability.

**Definition 4.3 • Bhattacharyya parameter**

For a binary-input channel $W$ with output alphabet $\mathcal{Y}$,

$$Z(W) = \sum_{y \in \mathcal{Y}} \sqrt{W(y \mid 0)\, W(y \mid 1)} \qquad \Big(\text{an integral for continuous } y\Big).$$

$Z(W) \in [0, 1]$. $Z = 0$: the two inputs give disjoint outputs (perfect channel). $Z = 1$: the outputs have the same distribution for both inputs (useless channel).

**Meaning.** $Z(W)$ is an upper bound on the error probability of the best (maximum-likelihood) decision about one bit sent over $W$: $P_e(W) \le Z(W)$. It also bounds capacity: $I(W) \ge \log_2 \frac{2}{1+Z(W)}$ and $I(W) \le \sqrt{1 - Z(W)^2}$.

**Worked Example 4.2 • Bhattacharyya parameters of the three channels**

**BEC($\epsilon$).** Only the output "?" has non-zero probability under both inputs: $Z = \sqrt{\epsilon \cdot \epsilon} = \epsilon$.
**BSC($p$).** $Z = \sqrt{(1-p)p} + \sqrt{p(1-p)} = 2\sqrt{p(1-p)}$. For $p = 0.1$: $Z = 0.6$.
**BI-AWGN.** Completing the square in the Gaussian integral gives $Z = \int \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y-1)^2+(y+1)^2}{4\sigma^2}} dy = e^{-1/(2\sigma^2)} = e^{-E_s/N_0}$. At $E_b/N_0 = 2\,\text{dB}$ and $R = 1/2$: $E_s/N_0 = 0.792$, $Z = e^{-0.792} = 0.453$.

### 4.5.1 Recursive reliability calculations

**Definition 4.4 • Bhattacharyya recursion (Arıkan)**

For any binary-input memoryless symmetric channel,

$$Z(W^+) = Z(W)^2, \qquad Z(W) \le Z(W^-) \le 2Z(W) - Z(W)^2,$$

with equality on the right for the BEC. Using the right-hand side as an approximation for all channels gives a simple (pessimistic) construction method.

```python
def bec_bhattacharyya(N, eps):
    """Z of the N bit-channels of a BEC(eps), natural index order."""
    if N == 1:
        return np.array([eps])
    zm = 2 * eps - eps ** 2              # minus channel (worse)
    zp = eps ** 2                        # plus  channel (better)
    return np.concatenate([bec_bhattacharyya(N // 2, zm),
                           bec_bhattacharyya(N // 2, zp)])
```

The function returns the first half for the minus branch and the second half for the plus branch, which matches the MSB-first rule in the note above. The cost is $O(N)$ operations.

## 4.6 Why do channels polarize?

**Intuition**

Each transform pushes a channel's capacity away from the middle: the plus child gains exactly what the minus child loses. A channel at $I = 0.5$ moves to 0.25 and 0.75; a channel at 0.9 moves to 0.81 and 0.99. Only channels already at 0 or 1 stay fixed. After many steps, nothing can stay in the middle.

**Advanced • The martingale argument**

Pick a random path: at each level choose minus or plus with probability 1/2. Let $I_n$ be the capacity of the channel reached after $n$ steps. Because $I(W^-) + I(W^+) = 2I(W)$,

$$\mathbb{E}[I_{n+1} \mid I_n] = \tfrac{1}{2} I(W^-) + \tfrac{1}{2} I(W^+) = I_n,$$

so $\{I_n\}$ is a bounded martingale and converges almost surely to a limit $I_\infty$. At the limit the transform must not move the capacity any more, which happens only for $I \in \{0, 1\}$. Since $\mathbb{E}[I_\infty] = I_0 = I(W)$, we get $P(I_\infty = 1) = I(W)$. This is exactly the polarization theorem: a fraction $I(W)$ of the channels becomes perfect. A similar argument on $Z_n$ (with $Z_{n+1} = Z_n^2$ half of the time) gives the speed of polarization, $Z_n \approx 2^{-2^{n/2}} = 2^{-\sqrt{N}}$.

**Professional insight**

The slow disappearance of the middle channels (Fig. 4.5) explains the *finite-length gap*: at $N = 1024$ a noticeable fraction of channels is neither good nor bad. Choosing them as

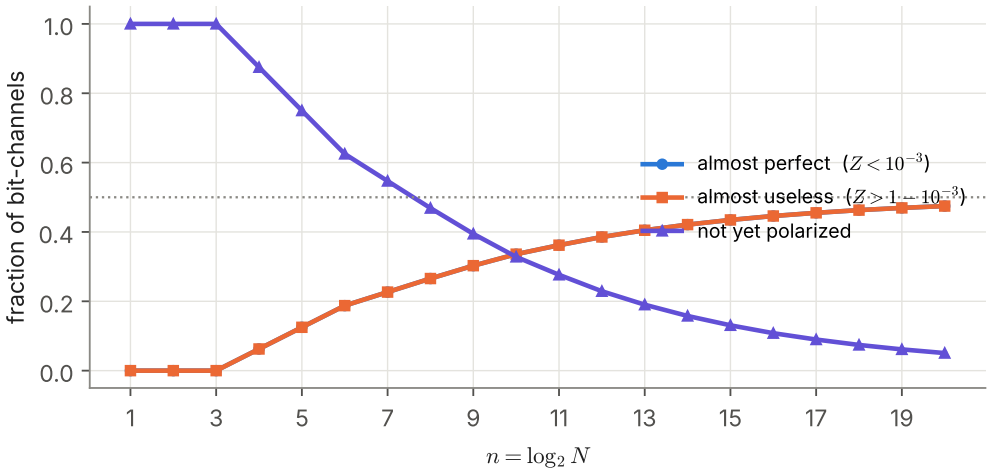



**Figure 4.5.** Fraction of polarized channels for BEC(0.5) as $N = 2^n$ grows. The "not yet polarized" fraction goes to zero, but slowly — even at $N = 2^{20} \approx 10^6$ it is not zero.

information bits causes errors; freezing them costs rate. Practical designs handle this with list decoding and CRC (Chapters 7–8) and with careful construction (Chapter 9).

## 4.7 Chapter summary

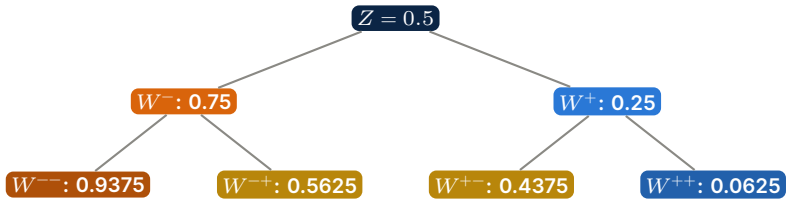
### Key concepts

- Combining: $W_N(\mathbf{y}|\mathbf{u}) = W^N(\mathbf{y}|\mathbf{u}\mathbf{G}_N)$, built recursively from two copies of $W_{N/2}$.
- Splitting: $W_N^{(i)}$ has input $u_i$ and output $(\mathbf{y}, u_0^{i-1})$; later bits are averaged out.
- One step creates a worse channel $W^-$ and a better channel $W^+$ with the same total capacity.
- On the BEC: $Z^- = 2Z - Z^2$, $Z^+ = Z^2$, exact. In general these give bounds.
- Reliability of index $i$: read $i$ in binary MSB first; 0 = minus, 1 = plus.

### Important formulas

| | |
|---|---|
| Bit-channel | $W_N^{(i)}(\mathbf{y}, u_0^{i-1}\|u_i) = \sum_{u_{i+1}^{N-1}} 2^{-(N-1)} W_N(\mathbf{y}\|\mathbf{u})$ |
| Conservation | $I(W^-) + I(W^+) = 2I(W)$ |
| Bhattacharyya | $Z(W) = \sum_y \sqrt{W(y\|0)W(y\|1)}$; BEC: $\epsilon$, BSC: $2\sqrt{p(1-p)}$, AWGN: $e^{-E_s/N_0}$ |
| Recursion | $Z^+ = Z^2$, $Z^- \le 2Z - Z^2$ (equality for BEC) |

**Visual recap • Chapter 4**




minus: $Z \to 2Z - Z^2$ plus: $Z \to Z^2$ indices $0, 1, 2, 3$ from left to right

## Common student mistakes

**! Common mistake**

- Applying the recursion LSB-first with the natural-order encoder of this book. With bit-reversed conventions (Arıkan's original $\mathbf{B}_N\mathbf{F}^{\otimes n}$) the rule changes; always check the convention.
- Thinking $Z(W^-) = 2Z - Z^2$ is exact for AWGN. It is only an upper bound outside the BEC.
- Confusing $Z$ (small is good) with $I$ (large is good) when ranking channels.

## Practice problems

**Problem 4.1.** For BEC(0.4), compute $Z$ for the four bit-channels of $N = 4$ and choose the information set for $K = 2$.

**Solution** $\epsilon^- = 0.64$, $\epsilon^+ = 0.16$. $Z_0 = 2(0.64) - 0.64^2 = 0.8704$, $Z_1 = 0.64^2 = 0.4096$, $Z_2 = 2(0.16) - 0.16^2 = 0.2944$, $Z_3 = 0.0256$. $\mathcal{A} = \{2, 3\}$. Check: capacities add to $4 \times 0.6 = 2.4$.

**Problem 4.2.** Compute $Z(W_8^{(5)})$ for BEC(0.5) using the binary rule.

**Solution** $5 = 101_2$: $0.5 \xrightarrow{+} 0.25 \xrightarrow{-} 0.4375 \xrightarrow{+} 0.1914$.

**Problem 4.3.** A BI-AWGN channel has $E_s/N_0 = 1$ (linear). Find $Z(W)$ and the upper bound for $Z(W^-)$.

**Solution** $Z = e^{-1} = 0.368$. $Z(W^-) \le 2(0.368) - 0.368^2 = 0.600$; $Z(W^+) = 0.135$.

**Problem 4.4.** Show that $I(W^-) + I(W^+) = 2I(W)$ holds for BEC($\epsilon$) using the erasure formulas.

**Solution** $(1 - 2\epsilon + \epsilon^2) + (1 - \epsilon^2) = 2 - 2\epsilon = 2(1 - \epsilon)$.

## Self-assessment quiz

1. Which bit sees the better channel in $W_2$: $u_0$ or $u_1$?
2. What extra information is included in the output of $W_N^{(i)}$?

3. For the BEC, is the minus channel also a BEC? With which erasure probability?
4. True or false: $Z(W^+) = Z(W)^2$ for every symmetric channel.

Answers: 1) $u_1$. 2) The previous bits $u_0, \ldots, u_{i-1}$. 3) Yes, $2\epsilon - \epsilon^2$. 4) True.

CHAPTER

# 5



# Polar Code Encoding

**IN THIS CHAPTER YOU WILL LEARN**

- the polar kernel $\mathbf{F}$ and the Kronecker product;
- how to write $\mathbf{G}_N = \mathbf{F}^{\otimes n}$ for any $N$;
- recursive and stage-by-stage (butterfly) encoding;
- frozen-bit placement and information-bit selection;
- complete $N = 4$ and $N = 8$ examples with every XOR;
- why the encoder costs only $\frac{N}{2}\log_2 N$ XORs.

## 5.1 The polar encoding kernel

**Definition 5.1 • Kernel**

The $2 \times 2$ **polar kernel** is

$$\mathbf{F} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \qquad [u_0\ u_1]\,\mathbf{F} = [\,u_0 \oplus u_1 \;\; u_1\,].$$

It is exactly the combining step of Fig. 4.1. Over GF(2), $\mathbf{F}\mathbf{F} = \mathbf{I}$, so the kernel is its own inverse.

**Intuition**

The kernel is the smallest non-trivial way to mix two bits: one output is the XOR of both, the other is a copy of one of them. Everything in polar coding is this little operation repeated.

## 5.2 The Kronecker product

**Definition 5.2 • Kronecker product**

For an $m \times n$ matrix $\mathbf{A}$ and any matrix $\mathbf{B}$,

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{bmatrix}.$$

Each entry of $\mathbf{A}$ is replaced by that entry times the whole matrix $\mathbf{B}$. The $n$-fold power is $\mathbf{F}^{\otimes n} = \mathbf{F} \otimes \mathbf{F}^{\otimes(n-1)}$ with $\mathbf{F}^{\otimes 1} = \mathbf{F}$.

Because $\mathbf{F}$ has a 0 in the top-right corner, the recursion has a very simple block form:

$$\mathbf{G}_N = \mathbf{F} \otimes \mathbf{G}_{N/2} = \begin{bmatrix} \mathbf{G}_{N/2} & \mathbf{0} \\ \mathbf{G}_{N/2} & \mathbf{G}_{N/2} \end{bmatrix}.$$

**Worked Example 5.1 • $\mathrm{G}_4$ and $\mathrm{G}_8$ step by step**

$$\mathbf{G}_4 = \mathbf{F} \otimes \mathbf{F} = \begin{bmatrix} 1\cdot\mathbf{F} & 0\cdot\mathbf{F} \\ 1\cdot\mathbf{F} & 1\cdot\mathbf{F} \end{bmatrix} = \left[\begin{array}{cc|cc} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ \hline 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{array}\right],$$

$$\mathbf{G}_8 = \begin{bmatrix} \mathbf{G}_4 & \mathbf{0} \\ \mathbf{G}_4 & \mathbf{G}_4 \end{bmatrix} = \left[\begin{array}{cccc|cccc} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ \hline 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{array}\right].$$

Row weights of $\mathbf{G}_8$: $1, 2, 2, 4, 2, 4, 4, 8$. In general the weight of row $i$ is $2^{w(i)}$, where $w(i)$ is the number of ones in the binary form of $i$.

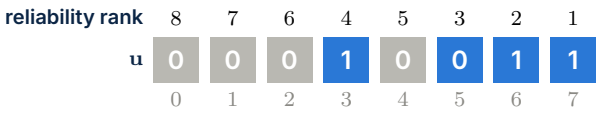
**Advanced • Row weights, minimum distance and Reed–Muller codes**

A polar code is spanned by the rows of $\mathbf{G}_N$ with indices in $\mathcal{A}$. Its minimum distance equals the smallest weight among those rows: $d_{\min} = \min_{i\in\mathcal{A}} 2^{w(i)}$. Reed–Muller codes use the same matrix but choose rows by *weight* (largest first); polar codes choose them by *bit-channel reliability*. For the $(8, 4)$ code with $\mathcal{A} = \{3, 5, 6, 7\}$ both rules agree and $d_{\min} = 4$; for long codes they differ. Polar codes have better SC performance, Reed–Muller codes larger distance; modern designs (e.g. PAC codes and 5G's sequence) mix both ideas.

## 5.3 Frozen-bit placement and information-bit selection

To encode we first build the length-$N$ vector $\mathbf{u}$:

1. compute a reliability for each index $i$ (Bhattacharyya parameter, GA, or a standard sequence);
2. put the $K$ message bits, in increasing index order, on the $K$ most reliable indices ($\mathcal{A}$);
3. set the other $N-K$ indices ($\mathcal{F}$) to the frozen value 0.




$K = 4$: the 4 most reliable indices $\{3,5,6,7\}$ receive the message $m = (1,0,1,1)$ in order: $u_3 = 1, u_5 = 0, u_6 = 1, u_7 = 1$.

**Important note**

The frozen value does not have to be 0 — any value known to both sides works, and 5G uses the freedom of "frozen" positions for parity-check bits in some cases (Chapter 10). For symmetric channels the error probability does not depend on the frozen values, so 0 is the standard choice.

The code itself is $\{\mathbf{u}\mathbf{G}_N : u_{\mathcal{F}} = \mathbf{0}\}$, i.e. the generator matrix of the $(N,K)$ code is the sub-matrix $\mathbf{G}_{\mathcal{A}}$ made of the rows of $\mathbf{G}_N$ with indices in $\mathcal{A}$: $\mathbf{x} = \mathbf{u}_{\mathcal{A}}\,\mathbf{G}_{\mathcal{A}}$.

## 5.4 Recursive encoding

The block form of $\mathbf{G}_N$ gives a recursion. Split $\mathbf{u} = [\mathbf{u}_a \,|\, \mathbf{u}_b]$ into halves:

$$\mathbf{x} = \mathbf{u}\mathbf{G}_N = [\mathbf{u}_a\ \mathbf{u}_b]\begin{bmatrix}\mathbf{G}_{N/2} & \mathbf{0}\\ \mathbf{G}_{N/2} & \mathbf{G}_{N/2}\end{bmatrix} = [(\mathbf{u}_a \oplus \mathbf{u}_b)\mathbf{G}_{N/2} \mid \mathbf{u}_b\mathbf{G}_{N/2}].$$

```python
def polar_encode_recursive(u):
    """x = u G_N with G_N = F^(x)n (no bit reversal)."""
    N = len(u)
    if N == 1:
        return u.copy()
    a, b = u[:N // 2], u[N // 2:]
    return np.concatenate([polar_encode_recursive(a ^ b),   # (u_a xor u_b) G_{N/2}
                           polar_encode_recursive(b)])      #  u_b G_{N/2}
```

## 5.5 Encoding stages and butterfly diagrams

Unrolling the recursion gives $n = \log_2 N$ **stages**. In stage $s$ ($s = 1, \ldots, n$) the bits are grouped in blocks of size $2h$ with $h = N/2^s$; inside each block the second half is XORed into the first half:

$$v_j \leftarrow v_j \oplus v_{j+h} \quad \text{for the upper positions } j \text{ of each block.}$$

Drawn as a graph, each stage looks like a set of crossing "butterfly" wings.

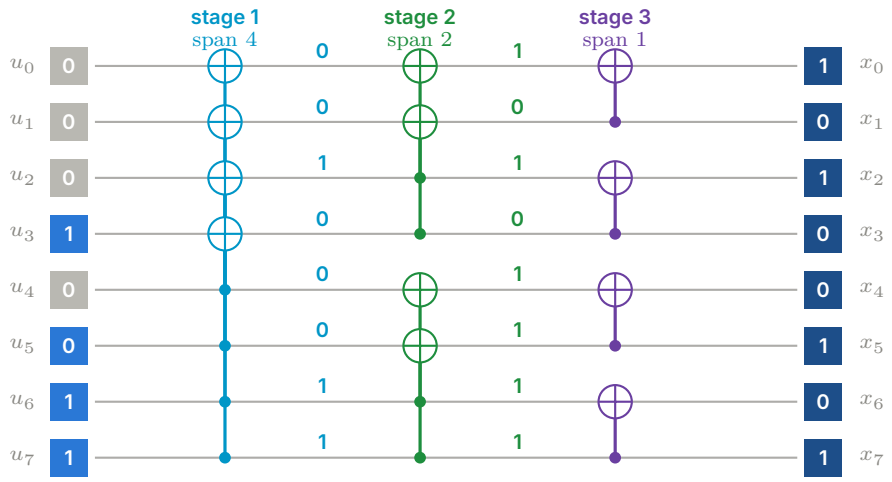



**Figure 5.1.** Butterfly encoder for $N = 8$ with the values of Worked Example 5.3. Small numbers are the intermediate vectors after stage 1 (cyan) and stage 2 (green). Each XOR node adds the bit coming up from the dot below.

**Worked Example 5.2 • $N = 4$ encoding, every XOR shown**

Code: $N = 4$, $K = 2$, BEC(0.5) ranking from Chapter 4 gives $\mathcal{A} = \{2, 3\}$, $\mathcal{F} = \{0, 1\}$. Message $m = (1, 1)$, so $\mathbf{u} = (0, 0, 1, 1)$.
**Stage 1 (span 2):** $v_0 = u_0 \oplus u_2 = 0 \oplus 1 = 1$, $v_1 = u_1 \oplus u_3 = 0 \oplus 1 = 1$, $v_2 = u_2 = 1$, $v_3 = u_3 = 1$.
**Stage 2 (span 1):** $x_0 = v_0 \oplus v_1 = 1 \oplus 1 = 0$, $x_1 = v_1 = 1$, $x_2 = v_2 \oplus v_3 = 1 \oplus 1 = 0$, $x_3 = v_3 = 1$.
**Result:** $\mathbf{x} = (0, 1, 0, 1)$.
**Check with the matrix:** rows 2 and 3 of $\mathbf{G}_4$ are 1010 and 1111; $1010 \oplus 1111 = 0101$.
**All four codewords** of this code: $m = 00 \to 0000$, $01 \to 1111$, $10 \to 1010$, $11 \to 0101$. Minimum distance 2.

**Worked Example 5.3 • $N = 8$ encoding, every XOR shown**

Code: $(8, 4)$, $\mathcal{A} = \{3, 5, 6, 7\}$, message $m = (1, 0, 1, 1)$, so $\mathbf{u} = (0, 0, 0, 1, 0, 0, 1, 1)$.
**Stage 1 (span 4)** — XOR $u_{j+4}$ into $u_j$ for $j = 0, 1, 2, 3$:

$$v_0 = u_0 \oplus u_4 = 0 \oplus 0 = 0 \qquad v_1 = u_1 \oplus u_5 = 0 \oplus 0 = 0$$
$$v_2 = u_2 \oplus u_6 = 0 \oplus 1 = 1 \qquad v_3 = u_3 \oplus u_7 = 1 \oplus 1 = 0$$
$$v_4 = u_4 = 0, \quad v_5 = u_5 = 0 \quad v_6 = u_6 = 1, \quad v_7 = u_7 = 1$$

$\mathbf{v} = (0, 0, 1, 0, 0, 0, 1, 1)$.
**Stage 2 (span 2)** — inside blocks $\{0..3\}$ and $\{4..7\}$:

$$w_0 = v_0 \oplus v_2 = 0 \oplus 1 = 1 \quad w_1 = v_1 \oplus v_3 = 0 \oplus 0 = 0 \quad w_2 = v_2 = 1 \quad w_3 = v_3 = 0$$
$$w_4 = v_4 \oplus v_6 = 0 \oplus 1 = 1 \quad w_5 = v_5 \oplus v_7 = 0 \oplus 1 = 1 \quad w_6 = v_6 = 1 \quad w_7 = v_7 = 1$$

$\mathbf{w} = (1, 0, 1, 0, 1, 1, 1, 1)$.
**Stage 3 (span 1)** — pairs $(0, 1), (2, 3), (4, 5), (6, 7)$:

$$x_0 = w_0 \oplus w_1 = 1 \quad x_1 = w_1 = 0 \quad x_2 = w_2 \oplus w_3 = 1 \quad x_3 = w_3 = 0$$
$$x_4 = w_4 \oplus w_5 = 0 \quad x_5 = w_5 = 1 \quad x_6 = w_6 \oplus w_7 = 0 \quad x_7 = w_7 = 1$$

**Result:** $\mathbf{x} = (1, 0, 1, 0, 0, 1, 0, 1)$, exactly as in Fig. 5.1.

**Check with the matrix:** $u_3 = u_6 = u_7 = 1$, so $\mathbf{x} = \text{row}_3 \oplus \text{row}_6 \oplus \text{row}_7 = 11110000 \oplus 10101010 \oplus 11111111 = 10100101$.

**Stage order does not matter**

Because XOR is linear and the stages act on different "bit positions" of the index, the stages may be executed in any order (span $1, 2, 4$ or $4, 2, 1$) and give the same $\mathbf{x}$. Hardware designers use this freedom.

## 5.6 Encoding complexity

Each stage performs $N/2$ XORs and there are $\log_2 N$ stages:

$$\text{XOR count} = \frac{N}{2}\log_2 N, \qquad \text{e.g. } N = 8: \ 12, \quad N = 1024: \ 5120.$$

A direct matrix product $\mathbf{u}\mathbf{G}_N$ would need up to $N^2/2 \approx 5 \times 10^5$ operations for $N = 1024$. The butterfly also shows that the encoder can be built with $\log_2 N$ pipeline stages of simple XOR gates.

```python
def polar_encode(u):
    """Butterfly encoder, works on a vector or on a (B, N) batch of vectors."""
    x = np.array(u, dtype=np.uint8, copy=True)
    one_d = (x.ndim == 1)
    if one_d:
        x = x[None, :]
    B, N = x.shape
    half = N // 2
    while half >= 1:                        # spans N/2, N/4, ..., 1
        x = x.reshape(B, -1, 2, half)       # blocks of 2*half bits
        x[:, :, 0, :] ^= x[:, :, 1, :]      # upper half ^= lower half
        x = x.reshape(B, N)
        half //= 2
    return x[0] if one_d else x
```

**Inverse encoding is the same circuit**

Since $\mathbf{G}_N\mathbf{G}_N = \mathbf{I}$ over GF(2), the same butterfly maps $\mathbf{x}$ back to $\mathbf{u} = \mathbf{x}\mathbf{G}_N$. Decoders use this to re-encode partial decisions (the "partial sums" of Chapter 6), and systematic encoders use it twice (Chapter 9).

## 5.7 Chapter summary

### Key concepts

- $\mathbf{G}_N = \mathbf{F}^{\otimes n} = \begin{bmatrix} \mathbf{G}_{N/2} & 0 \\ \mathbf{G}_{N/2} & \mathbf{G}_{N/2} \end{bmatrix}$.
- $\mathbf{u}$ holds message bits on $\mathcal{A}$ and zeros on $\mathcal{F}$; $\mathbf{x} = \mathbf{u}\mathbf{G}_N$.
- The encoder has $\log_2 N$ stages of $N/2$ XORs (butterflies).

- Row $i$ of $\mathbf{G}_N$ has weight $2^{w(i)}$; $d_{\min} = \min_{i \in \mathcal{A}} 2^{w(i)}$.

## Important formulas

| | |
|---|---|
| Kronecker recursion | $\mathbf{G}_N = \mathbf{F} \otimes \mathbf{G}_{N/2}$ |
| Recursive encoding | $\mathbf{x} = [(\mathbf{u}_a \oplus \mathbf{u}_b)\mathbf{G}_{N/2} \,\vert\, \mathbf{u}_b \mathbf{G}_{N/2}]$ |
| Stage rule | $v_j \leftarrow v_j \oplus v_{j+h}$, $h = N/2^s$ |
| Complexity | $\frac{N}{2} \log_2 N$ XORs |

**Visual recap • Chapter 5**




$\mathbf{u} = 0011 \Rightarrow \mathbf{x} = 0101$
$\frac{N}{2} \log_2 N = 4$ XORs

## Common student mistakes

**! Common mistake**

- Writing $\mathbf{F} = \left[\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right]$ and using row vectors — this is the transposed convention; results differ unless you also change the order of $\mathbf{u}$.
- Mixing the 5G (natural order) and Arıkan bit-reversed ($\mathbf{B}_N \mathbf{G}_N$) conventions in one design. The two give the same codes after re-indexing, but reliability tables are *not* interchangeable.
- Placing message bits on $\mathcal{A}$ in reliability order instead of index order — both sides must use the same mapping; this book (and 5G) uses increasing index order.

## Practice problems

**Problem 5.1.** Encode $\mathbf{u} = (1, 0, 1, 1)$ with $N = 4$, showing both stages.

**Solution** Stage 1: $v = (1 \oplus 1, 0 \oplus 1, 1, 1) = (0, 1, 1, 1)$. Stage 2: $x = (0 \oplus 1, 1, 1 \oplus 1, 1) = (1, 1, 0, 1)$. Matrix check: rows $0, 2, 3$: $1000 \oplus 1010 \oplus 1111 = 1101$.

**Problem 5.2.** For the $(8, 4)$ code with $\mathcal{A} = \{3, 5, 6, 7\}$, encode $m = (0, 1, 0, 1)$.

**Solution** $\mathbf{u} = (0, 0, 0, 0, 0, 1, 0, 1)$; rows 5 and 7: $11001100 \oplus 11111111 = 00110011$.

**Problem 5.3.** How many XORs does the butterfly encoder need for $N = 512$?

**Solution** $256 \times 9 = 2304$.

**Problem 5.4.** What is $d_{\min}$ of the polar code $N = 16$, $\mathcal{A} = \{7, 9, 10, 11, 12, 13, 14, 15\}$?

**Solution** Binary weights: $7 \to 3$, $9 \to 2$, $10 \to 2$, $11 \to 3$, $12 \to 2$, $13 \to 3$, $14 \to 3$, $15 \to 4$. Smallest row weight $2^2 = 4$, so $d_{\min} = 4$.

## Self-assessment quiz

1. What is $\mathbf{F} \otimes \mathbf{I}_2$? Is it equal to $\mathbf{I}_2 \otimes \mathbf{F}$?
2. What is the inverse of $\mathbf{G}_N$ over GF(2)?
3. In stage 1 of an $N = 16$ encoder, which bit is XORed into $v_3$?
4. True or false: the weight of row 12 of $\mathbf{G}_{16}$ is 4.

Answers: 1) $\left[\begin{smallmatrix}\mathbf{I} & 0\\ \mathbf{I} & \mathbf{I}\end{smallmatrix}\right]$; no, $\mathbf{I}_2 \otimes \mathbf{F} = \left[\begin{smallmatrix}\mathbf{F} & 0\\ 0 & \mathbf{F}\end{smallmatrix}\right]$. 2) $\mathbf{G}_N$ itself. 3) $u_{11}$. 4) True ($12 = 1100_2$, two ones).

PART III

# Advanced Level: Decoding Algorithms

Successive cancellation, list decoding, CRC aid, and the professional toolbox: fast decoders, construction methods and rate matching.

CHAPTER

# 6



# Successive Cancellation (SC) Decoding

**IN THIS CHAPTER YOU WILL LEARN**

- why a smart decoder is needed;
- likelihood ratios and log-likelihood ratios (LLRs);
- the $f$ and $g$ functions, derived from first principles;
- the SC decoding tree and its message flow;
- how frozen bits are handled;
- a complete $N = 8$ decoding example, number by number;
- complexity, latency, strengths and weaknesses of SC.

## 6.1 Why decoding is necessary

The receiver observes $\mathbf{y} = (y_0, \dots, y_{N-1})$, a noisy version of the codeword. The best possible decision — maximum likelihood (ML) — compares all $2^K$ codewords and picks the most likely one. For $K = 100$ this is $2^{100} \approx 10^{30}$ comparisons: impossible. A practical decoder must exploit the structure of the code. Arıkan's **successive cancellation** (SC) decoder uses the recursive structure of $\mathbf{G}_N$ and needs only $O(N \log N)$ operations.

## 6.2 The successive cancellation concept

**Intuition**

Remember the bit-channels: $W_N^{(i)}$ is a good channel *if the earlier bits are known*. So the decoder decides the bits one after another, $u_0, u_1, \dots, u_{N-1}$, and when deciding $u_i$ it pretends its own earlier decisions $\hat{u}_0, \dots, \hat{u}_{i-1}$ are correct. "Cancellation" means the effect of the already-decided bits is removed (cancelled) from the received signal before the next bit is decided — just like peeling layers off an onion.

**Definition 6.1 • SC decoding rule**

For $i = 0, 1, \dots, N-1$:

$$\hat{u}_i = \begin{cases} 0 \text{ (frozen value)} & i \in \mathcal{F}, \\ 0 & i \in \mathcal{A} \text{ and } L_N^{(i)}(\mathbf{y}, \hat{u}_0^{i-1}) \geq 0, \\ 1 & i \in \mathcal{A} \text{ and } L_N^{(i)}(\mathbf{y}, \hat{u}_0^{i-1}) < 0, \end{cases}$$

where $L_N^{(i)}$ is the LLR of the $i$-th bit-channel computed with the previous *decisions*.

## 6.3 Likelihood ratios and log-likelihood ratios

**Definition 6.2 • LR and LLR**

For a binary variable $c$ and an observation $y$,

$$\Lambda(y) = \frac{P(y \mid c=0)}{P(y \mid c=1)} \quad \text{(likelihood ratio)}, \qquad L(y) = \ln \Lambda(y) \quad \text{(log-likelihood ratio, LLR)}.$$

With equally likely bits, $L = \ln \frac{P(c=0|y)}{P(c=1|y)}$. The **sign** of $L$ is the hard decision ($L > 0 \Rightarrow 0$); the **magnitude** $|L|$ is the confidence.

### 6.3.1 LLR of BPSK over AWGN

With $s = 1 - 2c$ and $y = s + w$:

$$L(y) = \ln \frac{\exp(-\frac{(y-1)^2}{2\sigma^2})}{\exp(-\frac{(y+1)^2}{2\sigma^2})} = \frac{(y+1)^2 - (y-1)^2}{2\sigma^2} = \frac{4y}{2\sigma^2}.$$

$$L(y) = \frac{2y}{\sigma^2} \qquad (\text{BPSK}, 0 \mapsto +1,\ \sigma^2 = N_0/2).$$

The demodulator only scales $y$; the scale $2/\sigma^2$ tells the decoder how much to trust the channel.

**Worked Example 6.1 • From received value to LLR and probability**

$\sigma = 0.8$, $y = 0.4$. Then $L = 2(0.4)/0.64 = 1.25$, so $P(c=0|y)/P(c=1|y) = e^{1.25} = 3.49$ and $P(c = 0 \mid y) = 3.49/4.49 = 0.78$. A weak "0". With $y = -1.6$: $L = -5.0$, $P(c = 1 \mid y) = 0.993$ — a strong "1".

**Useful LLR facts**

- $P(c=0) = \dfrac{1}{1+e^{-L}}$, $P(c=1) = \dfrac{1}{1+e^{L}}$, and $\tanh(L/2) = P(c=0) - P(c=1)$.
- Independent observations of the same bit: LLRs *add*.
- The LLR of the XOR of two independent bits: tanh values *multiply* (next section).

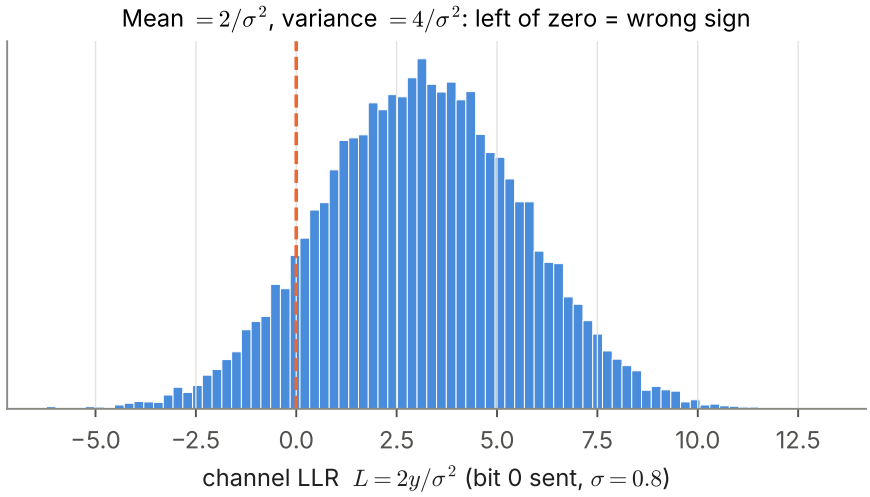



**Figure 6.1.** Histogram of channel LLRs when bit 0 is sent. Values left of zero would be hard-decision errors.

## 6.4 The f and g functions

Everything in SC decoding follows from decoding the basic $N = 2$ code: $x_0 = u_0 \oplus u_1$, $x_1 = u_1$, with channel LLRs $L_0$ (for $x_0$) and $L_1$ (for $x_1$).

### 6.4.1 Deciding $u_0$: the $f$ function

$u_0 = x_0 \oplus x_1$ and $u_1$ is unknown. Let $p_j = P(x_j = 0 \mid y_j)$. The XOR is 0 when both are 0 or both are 1:

$$P(u_0 = 0) = p_0 p_1 + (1 - p_0)(1 - p_1) \;\Rightarrow\; P(u_0 = 0) - P(u_0 = 1) = (2p_0 - 1)(2p_1 - 1).$$

Using $2p_j - 1 = \tanh(L_j/2)$:

$$L(u_0) = f(L_0, L_1) = 2\tanh^{-1}\Big(\tanh\tfrac{L_0}{2}\,\tanh\tfrac{L_1}{2}\Big) \;\approx\; \operatorname{sign}(L_0)\operatorname{sign}(L_1)\min\left(|L_0|, |L_1|\right).$$

The approximation is the **min-sum** rule. It is very accurate when one input is much larger than the other, never wrong in sign, and slightly overestimates the magnitude.

### 6.4.2 Deciding $u_1$ when $u_0$ is known: the $g$ function

Now $u_1 = x_1$ is seen directly through $L_1$, and also through $x_0 = u_0 \oplus u_1$: if $u_0 = 0$, $L_0$ is another view of $u_1$; if $u_0 = 1$, it is a view of $\bar{u}_1$, so its sign flips. Independent views add:

$$L(u_1) = g(L_0, L_1, \hat{u}_0) = L_1 + (1 - 2\hat{u}_0)\, L_0.$$

**Worked Example 6.2 • $f$ and $g$ on numbers**

$L_0 = +1.2$, $L_1 = -0.4$.
**Exact $f$:** $\tanh(0.6) = 0.5370$, $\tanh(-0.2) = -0.1974$, product $-0.1060$, $f = 2\tanh^{-1}(-0.1060) = -0.2128$. Check with probabilities: $p_0 = 0.7685$, $p_1 = 0.4013$,

$P(u_0 = 0) = 0.4470$, $\ln(0.4470/0.5530) = -0.2128$.
**Min-sum $f$:** $(+)(-)\min(1.2, 0.4) = -0.4$ (same sign, larger magnitude).
**$g$:** if $\hat{u}_0 = 0$, $g = -0.4 + 1.2 = +0.8$; if $\hat{u}_0 = 1$, $g = -0.4 - 1.2 = -1.6$.

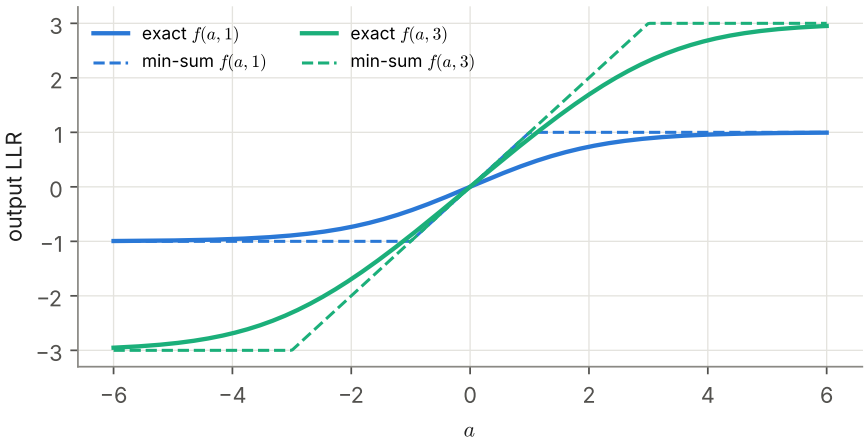



**Figure 6.2.** Exact $f$ (solid) and its min-sum approximation (dashed) for $L_1 = 1$ and $L_1 = 3$. The output can never be more reliable than the less reliable input.

## 6.5 The SC decoding tree

For general $N$ the same two operations are applied recursively. Recall the recursive encoder: $\mathbf{x} = [(\mathbf{u}_a \oplus \mathbf{u}_b)\mathbf{G}_{N/2} \,|\, \mathbf{u}_b\mathbf{G}_{N/2}]$. Writing $\mathbf{c}_a = \mathbf{u}_a\mathbf{G}_{N/2}$ and $\mathbf{c}_b = \mathbf{u}_b\mathbf{G}_{N/2}$, the left half of $\mathbf{x}$ is $\mathbf{c}_a \oplus \mathbf{c}_b$ and the right half is $\mathbf{c}_b$ — exactly the $N = 2$ code applied element by element. Hence:

**Definition 6.3 • SC node operation**

A node of size $n$ receives an LLR vector $\alpha$ of length $n$ (split into halves $\alpha_a, \alpha_b$) and returns a bit vector $\beta$ (its re-encoded decisions, the **partial sums**):

1. left child input: $\alpha_L = f(\alpha_a, \alpha_b)$ (element-wise); it returns $\beta_L$;
2. right child input: $\alpha_R = g(\alpha_a, \alpha_b, \beta_L)$; it returns $\beta_R$;
3. output: $\beta = [\beta_L \oplus \beta_R \,|\, \beta_R]$.

A leaf (size 1) is one bit $u_i$: it decides $\hat{u}_i$ by the rule above and returns $\beta = \hat{u}_i$.

## 6.6 Hard decisions and frozen-bit handling

At a leaf:

- **information bit** ($i \in \mathcal{A}$): $\hat{u}_i = 0$ if $L \geq 0$, else 1 (hard decision on the bit-channel LLR);
- **frozen bit** ($i \in \mathcal{F}$): $\hat{u}_i = 0$ *whatever* $L$ says. The LLR is still computed (it is needed for nothing else) — but the decoder trusts its prior knowledge.

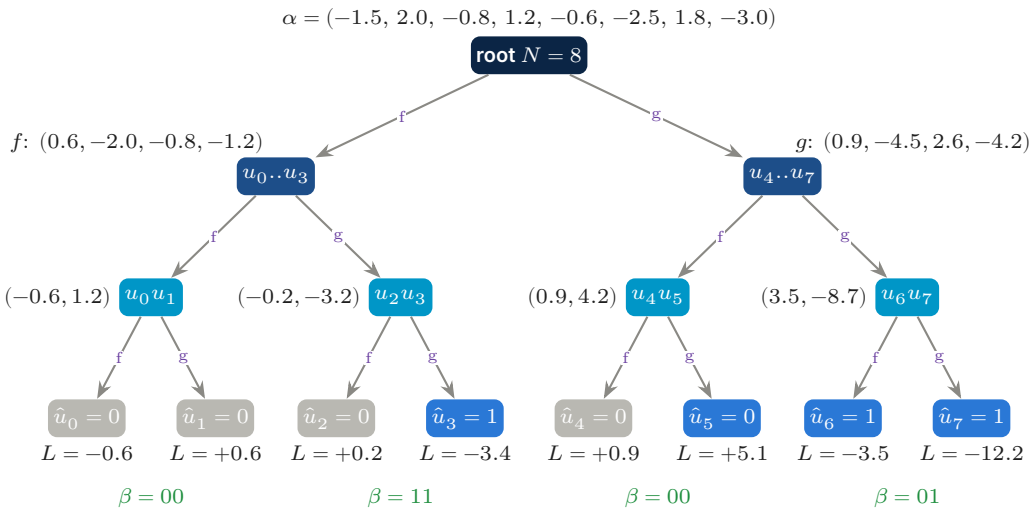



**Figure 6.3.** SC decoding tree for $N = 8$ with the numbers of Worked Example 6.3. LLRs travel down ($f$ to the left child, $g$ to the right child); partial sums $\beta$ travel up. Leaves are visited left to right: $u_0, u_1, \ldots, u_7$. Grey leaves are frozen, blue leaves are information bits.

**! Do not "decode" a frozen bit**

A frozen leaf may have a negative LLR (in our example $u_0$ has $L = -0.6$). Setting $\hat{u}_0 = 1$ because of the sign would be a serious bug: the partial sums would be wrong and every later decision would be corrupted. Frozen bits are *known*; this certainty is where the coding gain comes from.

## 6.7 Step-by-step numerical decoding example

**Worked Example 6.3 • Complete SC decoding, $N = 8$, $K = 4$**

**Setting.** The $(8, 4)$ code with $\mathcal{F} = \{0, 1, 2, 4\}$ sends $m = (1, 0, 1, 1)$, i.e. $\mathbf{u} = (0, 0, 0, 1, 0, 0, 1, 1)$ and $\mathbf{x} = (1, 0, 1, 0, 0, 1, 0, 1)$ (Chapter 5). BPSK gives $\mathbf{s} = (-1, +1, -1, +1, +1, -1, +1, -1)$. The received LLRs are

$$\alpha = (-1.5,\ 2.0,\ -0.8,\ 1.2,\ \mathbf{-0.6},\ -2.5,\ 1.8,\ -3.0).$$

Position 4 has the wrong sign (sent $+1$): a hard-decision receiver would make one error. We use min-sum $f$.

**Step 1 — root, $f$ to the left child** ($\alpha_a = (-1.5, 2.0, -0.8, 1.2)$, $\alpha_b = (-0.6, -2.5, 1.8, -3.0)$):

$$f(-1.5, -0.6) = +0.6, \quad f(2.0, -2.5) = -2.0, \quad f(-0.8, 1.8) = -0.8, \quad f(1.2, -3.0) = -1.2.$$

**Step 2 — node $u_0..u_3$, $f$ to node $u_0u_1$:** $f(0.6, -0.8) = -0.6$, $f(-2.0, -1.2) = +1.2$.
**Step 3 — leaf $u_0$:** $L = f(-0.6, 1.2) = -0.6$. Frozen $\Rightarrow \hat{u}_0 = 0$.
**Step 4 — leaf $u_1$:** $L = g(-0.6, 1.2, 0) = 1.2 + (-0.6) = +0.6$. Frozen $\Rightarrow \hat{u}_1 = 0$. Node $u_0u_1$ returns $\beta = (\hat{u}_0 \oplus \hat{u}_1, \hat{u}_1) = (0, 0)$.
**Step 5 — $g$ to node $u_2u_3$** with $\beta_L = (0, 0)$: $g(0.6, -0.8, 0) = -0.2$, $g(-2.0, -1.2, 0) = -3.2$.
**Step 6 — leaf $u_2$:** $L = f(-0.2, -3.2) = +0.2$. Frozen $\Rightarrow \hat{u}_2 = 0$.
**Step 7 — leaf $u_3$:** $L = g(-0.2, -3.2, 0) = -3.4$. Information, $L < 0 \Rightarrow \hat{u}_3 = 1$. Node returns $\beta = (0 \oplus 1, 1) = (1, 1)$.

**Step 8 — node $u_0..u_3$ returns** $[\beta_L \oplus \beta_R \,|\, \beta_R] = [(0,0) \oplus (1,1) \,|\, (1,1)] = (1,1,1,1)$.
**Step 9 — root, $g$ to the right child** with $\beta_L = (1,1,1,1)$, so $g = \alpha_b - \alpha_a$:

$$-0.6 + 1.5 = 0.9, \quad -2.5 - 2.0 = -4.5, \quad 1.8 + 0.8 = 2.6, \quad -3.0 - 1.2 = -4.2.$$

**Step 10 — node $u_4..u_7$, $f$ to node $u_4u_5$:** $f(0.9, 2.6) = 0.9$, $f(-4.5, -4.2) = 4.2$.
**Step 11 — leaf $u_4$:** $L = f(0.9, 4.2) = 0.9$. Frozen $\Rightarrow \hat{u}_4 = 0$.
**Step 12 — leaf $u_5$:** $L = g(0.9, 4.2, 0) = 5.1$. Information $\Rightarrow \hat{u}_5 = 0$. Node returns $(0,0)$.
**Step 13 — $g$ to node $u_6u_7$:** the node inputs are $\alpha_a = (0.9, -4.5)$ and $\alpha_b = (2.6, -4.2)$ with $\beta_L = (0,0)$, so $g(0.9, 2.6, 0) = 3.5$ and $g(-4.5, -4.2, 0) = -8.7$.
**Step 14 — leaf $u_6$:** $L = f(3.5, -8.7) = -3.5 \Rightarrow \hat{u}_6 = 1$.
**Step 15 — leaf $u_7$:** $L = g(3.5, -8.7, 1) = -8.7 - 3.5 = -12.2 \Rightarrow \hat{u}_7 = 1$. Node returns $(0,1)$.
**Step 16 — results.** $\hat{\mathbf{u}} = (0,0,0,1,0,0,1,1)$, message $\hat{m} = (\hat{u}_3, \hat{u}_5, \hat{u}_6, \hat{u}_7) = (1,0,1,1)$ — **correct**. The root returns $\hat{\mathbf{x}} = (1,0,1,0,0,1,0,1)$: the channel error at position 4 has been corrected.

**Watch the reliabilities grow**

The information bits were decided with $|L| = 3.4, 5.1, 3.5, 12.2$, much larger than most channel LLRs. The frozen bits had small or even negative LLRs $(-0.6, 0.6, 0.2, 0.9)$ — exactly the polarization predicted in Chapter 4.

## 6.8 A simple SC decoder in Python

This version mirrors the tree exactly and is written for clarity (the fast, vectorized version is in Chapter 11).

```python
def f(a, b):                        # min-sum check-node rule
    return np.sign(a) * np.sign(b) * np.minimum(np.abs(a), np.abs(b))

def g(a, b, c):                     # c = partial sums from the left child
    return b + (1 - 2 * c) * a

def sc_node(alpha, frozen, start, u_hat):
    """Decode the leaves start..start+len(alpha)-1; return the partial sums beta."""
    n = len(alpha)
    if n == 1:                                      # leaf: one bit u_start
        bit = 0 if frozen[start] else int(alpha[0] < 0)
        u_hat[start] = bit
        return np.array([bit])
    h = n // 2
    a, b = alpha[:h], alpha[h:]
    beta_L = sc_node(f(a, b), frozen, start, u_hat)           # left  child
    beta_R = sc_node(g(a, b, beta_L), frozen, start + h, u_hat) # right child
    return np.concatenate([beta_L ^ beta_R, beta_R])

def sc_decode(llr, frozen):
    u_hat = np.zeros(len(llr), dtype=int)
    sc_node(np.asarray(llr, float), frozen, 0, u_hat)
```

```
    return u_hat

frozen = np.array([1, 1, 1, 0, 1, 0, 0, 0], bool)       # F = {0,1,2,4}
print(sc_decode([-1.5, 2.0, -0.8, 1.2, -0.6, -2.5, 1.8, -3.0], frozen))
```

```
OUTPUT
[0 0 0 1 0 0 1 1]
```

## 6.9 Computational complexity

- At each of the $\log_2 N$ tree levels the decoder computes $N/2$ $f$ values and $N/2$ $g$ values in total (level with node size $n$: $N/n$ nodes $\times$ $n/2$ $f$ and $n/2$ $g$). Total: $N \log_2 N$ LLR operations, i.e. $O(N \log N)$.
- Partial sums: also $\frac{N}{2} \log_2 N$ XORs (it is a re-encoding).
- Memory: one LLR vector per tree level suffices if the nodes are visited depth-first: $N + N/2 + \cdots + 1 \approx 2N$ LLRs, plus $\approx 2N$ partial-sum bits: $O(N)$.
- Latency: a straightforward hardware tree decoder needs $2N - 2$ clock cycles (one per node visit when each level computes in one cycle). Fast simplifications reduce this a lot (Chapter 9).

**Advanced • Why SC is exactly "bit-channel" decoding**

One can prove that the LLR computed at leaf $i$ by the recursion (with exact $f$) equals $\ln \frac{W_N^{(i)}(\mathbf{y}, \hat{u}_0^{i-1} | 0)}{W_N^{(i)}(\mathbf{y}, \hat{u}_0^{i-1} | 1)}$, the LLR of the synthetic channel of Chapter 4. We verified this numerically: for $N = 8$ and 200 random LLR vectors, the decoder in Chapter 11 reproduced the brute-force bit-channel LLRs (summing over all $2^{N-i-1}$ future bit patterns) for every leaf.

## 6.10 Advantages and limitations

| Advantages | Limitations |
|---|---|
| $O(N \log N)$ complexity, tiny memory | **Error propagation**: one wrong decision corrupts all later partial sums |
| Achieves capacity as $N \to \infty$ | Weak at short and medium lengths |
| Simple, regular hardware ($f$, $g$, XOR) | Inherently sequential: bit $i$ waits for bits $0..i-1$ |
| Only needs LLRs and the frozen set | No soft output (hard decisions only) |

**Professional insight**

Error propagation is the key weakness. An SC decoder never reconsiders a decision: if $\hat{u}_3$ is wrong, the $g$ functions for $u_4, \ldots, u_7$ use wrong partial sums. The cure is to keep *several* candidate decisions alive — the list decoder of the next chapter.

## 6.11 Chapter summary

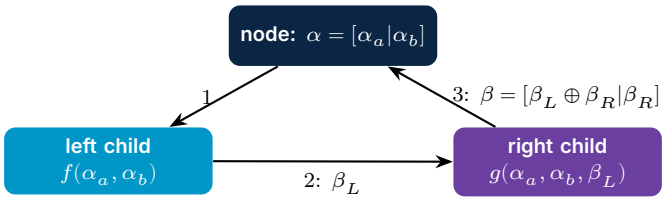
### Key concepts

- SC decodes $u_0, \dots, u_{N-1}$ in order, using its own earlier decisions as if correct.
- BPSK/AWGN channel LLR: $L = 2y/\sigma^2$.
- $f$ combines two LLRs of an XOR; $g$ adds two views of the same bit after cancelling the known bit.
- The tree sends LLRs down and partial sums up; frozen leaves always decide 0.
- Complexity $O(N \log N)$, memory $O(N)$, but sequential and sensitive to error propagation.

### Important formulas

| | |
|---|---|
| Channel LLR | $L = 2y/\sigma^2$ |
| $f$ (exact / min-sum) | $2\tanh^{-1}(\tanh\frac{a}{2}\tanh\frac{b}{2}) \approx \operatorname{sign}(a)\operatorname{sign}(b)\min(\lvert a\rvert, \lvert b\rvert)$ |
| $g$ | $b + (1-2c)\,a$ |
| Partial sums | $\beta = [\beta_L \oplus \beta_R \,\vert\, \beta_R]$ |
| Complexity | $N \log_2 N$ $f/g$ operations |

**Visual recap • Chapter 6**




leaf: frozen $\to 0$; info $\to$ sign of $L$
cost $O(N \log N)$; error propagation!

### Common student mistakes

**! Common mistake**

- Feeding the decoder $y$ instead of $L = 2y/\sigma^2$. With min-sum $f$ the decisions do not change under a positive scale factor, but the exact $f$, exact list metrics (Chapter 7) and fixed-point hardware need correct scaling.
- Using $u$ bits instead of partial sums $\beta$ in $g$. The $g$ function needs the re-encoded left child, e.g. $(\hat{u}_0 \oplus \hat{u}_1, \hat{u}_1)$, not $(\hat{u}_0, \hat{u}_1)$.
- Mixing the order of halves: with this book's convention, $f$ and $g$ combine element $j$ of the first half with element $j$ of the second half.
- Making a hard decision on frozen bits.

## Practice problems

**Problem 6.1.** Compute exact and min-sum $f(2.0, -3.0)$.

**Solution** $\tanh(1) = 0.7616$, $\tanh(-1.5) = -0.9051$, product $-0.6893$, $f = 2\tanh^{-1}(-0.6893) = -1.693$. Min-sum: $-2.0$.

**Problem 6.2.** $N = 2$, $K = 1$, $\mathcal{A} = \{1\}$. Received LLRs $(L_0, L_1) = (-0.5, +0.3)$. Decode with SC.

**Solution** $u_0$ frozen $= 0$. $L(u_1) = g(-0.5, 0.3, 0) = -0.2 \Rightarrow \hat{u}_1 = 1$, $\hat{\mathbf{x}} = (1, 1)$. This is the repetition code: the sum of the two LLRs decides.

**Problem 6.3.** $N = 4$, $\mathcal{A} = \{2, 3\}$, LLRs $(1.0, -2.0, -1.5, 0.5)$. Decode with min-sum SC.

**Solution** $f$ to left: $f(1.0, -1.5) = -1.0$, $f(-2.0, 0.5) = -0.5$. $u_0$: frozen. $u_1$: frozen; $\beta_L = (0, 0)$. $g$ to right: $(-1.5 + 1.0,\ 0.5 - 2.0) = (-0.5, -1.5)$. $u_2$: $f(-0.5, -1.5) = +0.5 \Rightarrow \hat{u}_2 = 0$ (two negative inputs give a positive XOR!). $u_3$: $g(-0.5, -1.5, 0) = -1.5 - 0.5 = -2.0 \Rightarrow \hat{u}_3 = 1$. $\hat{\mathbf{u}} = (0, 0, 0, 1)$, $\hat{\mathbf{x}} = (1, 1, 1, 1)$. Verified with the code above.

**Problem 6.4.** How many $f$ and $g$ evaluations does SC need for $N = 1024$?

**Solution** $\frac{N}{2}\log_2 N = 5120$ of each, 10240 in total.

## Self-assessment quiz

1. What does the sign of an LLR mean? And its magnitude?
2. Why can the magnitude of $f(a, b)$ never exceed $\min(|a|, |b|)$?
3. What information flows up the SC tree?
4. Why is SC decoding hard to parallelize?

Answers: 1) Sign = hard decision, magnitude = confidence. 2) An XOR is only as reliable as its weakest input. 3) Partial sums (re-encoded decisions). 4) Each bit needs all earlier decisions.

CHAPTER

# 7



# Successive Cancellation List (SCL) Decoding

**IN THIS CHAPTER YOU WILL LEARN**

- a real example where SC fails;
- path splitting and the list size $L$;
- the LLR-based path metric and its derivation;
- how candidates are pruned;
- a complete $L = 2$ example that corrects SC's mistake;
- SCL versus SC: simulation results;
- complexity and memory.

## 7.1 Why SC decoding is sometimes insufficient

**Intuition**

A chess player who must commit to every move without ever looking ahead will sometimes play a move that looks good now and is a disaster three moves later. SC is like this: it commits to each bit at once. A *list* decoder looks ahead a little: at every uncertain bit it keeps both possibilities alive and lets the *future* — in particular the frozen bits, whose values are known — decide which was right.

**Worked Example 7.1 • SC makes a wrong decision**

Same $(8, 4)$ code and same transmitted codeword as Worked Example 6.3 ($\mathbf{u} = (0, 0, 0, 1, 0, 0, 1, 1)$, $\mathbf{s} = (-1, +1, -1, +1, +1, -1, +1, -1)$), but a different noise realization:

$$\alpha = (-2.0,\ 0.3,\ -0.4,\ \mathbf{-2.0},\ 1.0,\ -5.8,\ 3.2,\ -4.8).$$

Only position 3 has the wrong sign, but it is a *strong* error ($-2.0$). Running SC:

| $u_0$ | $u_1$ | $u_2$ | $u_3$ | $u_4$ | $u_5$ | $u_6$ | $u_7$ |
|---|---|---|---|---|---|---|---|
| frozen | frozen | frozen | info | frozen | info | info | info |
| $-0.3$ | $+0.1$ | $-1.4$ | $\mathbf{+0.3}$ | $\mathbf{-1.0}$ | $+4.5$ | $-1.8$ | $-14.1$ |
| 0 | 0 | 0 | **0** × | 0 | 0 | 1 | 1 |

At $u_3$ the LLR is $+0.3$: weak, but positive, so SC decides $\hat{u}_3 = 0$ — wrong. The decision is never reconsidered. Note the warning sign one step later: frozen bit $u_4$ (known to be 0) gets LLR $-1.0$, i.e. the decoder "believes" $u_4 = 1$. SC ignores this hint. The output $\hat{m} = (0, 0, 1, 1)$ is a block error.

## 7.2 Path splitting

**Definition 7.1 • Decoding path**

A **decoding path** is a candidate partial vector $\hat{u}_0^i = (\hat{u}_0, \dots, \hat{u}_i)$ together with its own copy of the internal decoder state (LLRs and partial sums) and a reliability score, the **path metric**.

At an information bit, each path **splits** into two children: one with $\hat{u}_i = 0$, one with $\hat{u}_i = 1$. At a frozen bit, each path has only one child ($\hat{u}_i = 0$). Without control the number of paths would double at every information bit and reach $2^K$ — this is ML decoding, which is too expensive.

**Definition 7.2 • SCL decoder**

The **successive cancellation list** decoder of list size $L$ runs SC on up to $L$ paths in parallel. After each split it keeps only the $L$ paths with the best (smallest) path metrics. At the end it outputs the best path (or, with a CRC, the best path that passes the CRC — Chapter 8).

## 7.3 Path metrics

### 7.3.1 Derivation

For path $l$ with decisions $\hat{u}_0^i[l]$ we would like to rank by the probability $P(\hat{u}_0^i[l] \mid \mathbf{y})$. Following Balatsoukas-Stimming, Bastani Parizi and Burg (2015), define the path metric as the negative log of this probability (up to a common constant). Using the chain rule,

$$P(\hat{u}_0^i \mid \mathbf{y}) = \prod_{j=0}^{i} P(\hat{u}_j \mid \mathbf{y}, \hat{u}_0^{j-1}), \qquad P(\hat{u}_j \mid \mathbf{y}, \hat{u}_0^{j-1}) = \frac{1}{1 + e^{-(1-2\hat{u}_j)L_j}},$$

where $L_j$ is the bit-channel LLR computed on that path. Taking $-\ln$:

$$\mathrm{PM}_l^{(i)} = \sum_{j=0}^{i} \ln\left(1 + e^{-(1-2\hat{u}_j[l])\,L_j[l]}\right) \approx \sum_{j=0}^{i} \begin{cases} 0 & \text{if } \hat{u}_j \text{ agrees with } \operatorname{sign}(L_j), \\ |L_j| & \text{otherwise.} \end{cases}$$

The approximation uses $\ln(1 + e^x) \approx \max(0, x)$. It is the form used in this book and in most hardware.

**A penalty system**

Each path starts with $\mathrm{PM} = 0$. Whenever a path takes a decision *against* the LLR, it pays a penalty equal to $|L|$ — the confidence of the LLR it contradicted. Contradicting a weak LLR is cheap; contradicting a strong one is expensive. Frozen bits also cost a penalty when their LLR says "1": this is how a wrong path gets punished later.

### 7.3.2 Update rule in practice

For path $l$ at bit $i$ with LLR $L$:

- frozen: PM $\leftarrow$ PM $+$ ($|L|$ if $L < 0$ else 0);
- information, child $\hat{u}_i = 0$: PM $+$ ($|L|$ if $L < 0$ else 0);
- information, child $\hat{u}_i = 1$: PM $+$ ($|L|$ if $L > 0$ else 0).

The child that follows the LLR keeps the parent's metric; the other child pays $|L|$.

## 7.4 Candidate pruning

After splitting, there are up to $2L$ candidates. The decoder sorts their metrics and keeps the $L$ smallest; the others are deleted. Two details matter:

1. At the start there is one path. For the first $\log_2 L$ information bits the list grows $(1, 2, 4, \dots)$ and no pruning is necessary.
2. When a parent has *both* children kept, its state must be copied. When neither child is kept, the parent's memory is freed. Tal and Vardy's *lazy copy* shares memory between paths until a copy is really needed, keeping the complexity at $O(LN \log N)$.

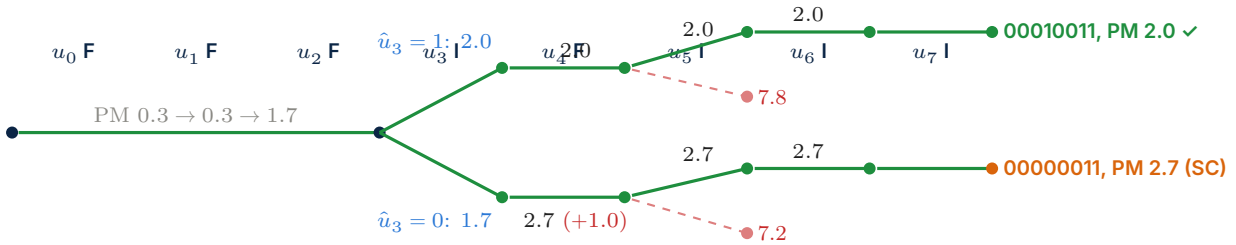



**Figure 7.1.** The surviving paths of the $L = 2$ decoder of Worked Example 7.2 (pruned candidates at $u_5$ shown dashed; at $u_6$ and $u_7$ the pruned candidates have metrics above 4.5 and are not drawn). The SC path (bottom) pays 1.0 at frozen bit $u_4$ and loses.

## 7.5 Worked SCL decoding example

**Worked Example 7.2 • SCL with $L = 2$ corrects the SC error**

Same received LLRs as Worked Example 7.1. For each candidate we show (decisions, PM, LLR of the bit on that path).

| bit | type | candidates after the update (kept, pruned) |
|---|---|---|
| $u_0$ | F | 0 (0.3) $L = -0.3$, frozen decided 0 against the sign: +0.3 |
| $u_1$ | F | 00 (0.3) $L = +0.1$ |
| $u_2$ | F | 000 (1.7) $L = -1.4$: +1.4 |
| $u_3$ | I | $L = +0.3$: 0000 (1.7), 0001 (2.0) — only 2 candidates, both kept |
| $u_4$ | F | path 0001: $L = +2.8 \Rightarrow$ 00010 (2.0); path 0000: $L = -1.0 \Rightarrow$ 00000 (2.7) |
| $u_5$ | I | path 00010 ($L = +5.8$): 000100 (2.0), 000101 (7.8); path 00000 ($L = +4.5$): 000000 (2.7), 000001 (7.2) |
| $u_6$ | I | path 000100 ($L = -6.6$): 0001001 (2.0), 0001000 (8.6); path 000000 ($L = -1.8$): 0000001 (2.7), 0000000 (4.5) |
| $u_7$ | I | path 0001001 ($L = -15.5$): 00010011 (2.0), 00010010 (17.5); path 0000001 ($L = -14.1$): 00000011 (2.7), 00000010 (16.8) |

**Result.** The best final path is $\hat{\mathbf{u}} = (0, 0, 0, 1, 0, 0, 1, 1)$ with PM = 2.0: message $(1, 0, 1, 1)$ — **correct**. The SC path $(0, 0, 0, 0, 0, 0, 1, 1)$ ends with PM = 2.7.

**Why it worked.** At $u_3$ the wrong choice was *slightly* cheaper (1.7 vs 2.0). But on the wrong path the frozen bit $u_4$ had $L = -1.0$ and cost a penalty of 1.0, while on the correct path it had $L = +2.8$ and cost nothing. The frozen bit acted as a free parity check. This is the essence of list decoding.

**At the end, PM is a distance to the received signal**

With the approximate metric, the final PM of a complete path equals the sum of $|\alpha_j|$ over the channel positions where its codeword $\hat{\mathbf{u}}\mathbf{G}_N$ disagrees with the hard decisions $\text{sign}(\alpha_j)$ (the *correlation discrepancy*). Here the hard decisions are $1, 0, 1, 1, 0, 1, 0, 1$.

- Correct path: codeword 10100101 differs only at position 3: PM $= |-2.0| = 2.0$
- SC path: codeword 01010101 differs at positions $0, 1, 2$: PM $= 2.0 + 0.3 + 0.4 = 2.7$

So with a large list the decoder picks the codeword closest to the received signal: SCL approaches ML decoding.

## 7.6 SCL versus SC performance

| BLER at $E_b/N_0 =$ | 2.0 dB | 2.5 dB | 3.0 dB | 3.5 dB |
|---|---|---|---|---|
| SC | $1.3 \times 10^{-1}$ | $4.6 \times 10^{-2}$ | $1.2 \times 10^{-2}$ | $2.7 \times 10^{-3}$ |
| SCL, $L = 2$ | $6.5 \times 10^{-2}$ | $1.8 \times 10^{-2}$ | $4.6 \times 10^{-3}$ | $1.4 \times 10^{-3}$ |
| SCL, $L = 4$ | $4.5 \times 10^{-2}$ | $1.3 \times 10^{-2}$ | $3.7 \times 10^{-3}$ | $1.3 \times 10^{-3}$ |
| SCL, $L = 8$ | $4.2 \times 10^{-2}$ | $1.2 \times 10^{-2}$ | $3.7 \times 10^{-3}$ | $1.3 \times 10^{-3}$ |

**Reading the simulation**

- $L = 2$ already gains about 0.3–0.4 dB over SC at BLER $10^{-2}$.
- Going from $L = 4$ to $L = 8$ gives almost nothing. The list decoder is already close to ML decoding of *this* code; the remaining errors are codewords that are genuinely

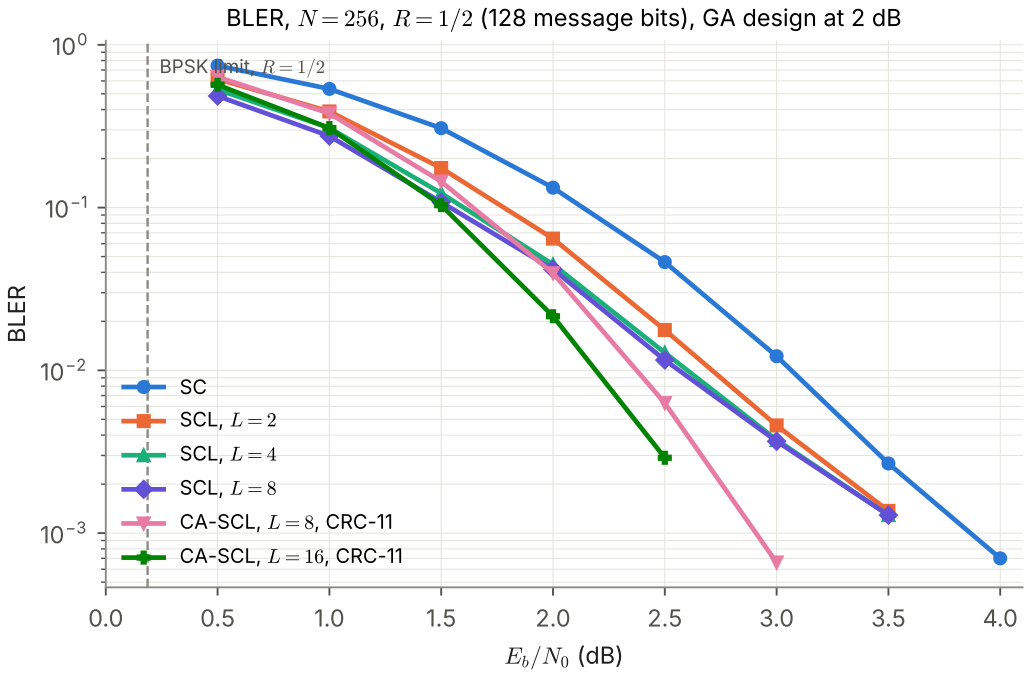



**Figure 7.2.** Monte-Carlo BLER of the $(256, 128)$ polar code over BPSK-AWGN (code of Chapter 11, GA construction at 2 dB, min-sum metrics, at least 150 block errors per point where possible).

closer to the received signal — the code's small minimum distance is the limit, not the decoder.

- The way forward is not a larger list but a better *code*: adding a CRC (next chapter) gives a large extra gain (green and pink curves).

## 7.7 Complexity and memory requirements

| | |
|---|---|
| **Computation** | $O(LN \log N)$: $L$ SC decoders in parallel (with lazy copy). |
| **Sorting** | at each of the $K$ information bits, select the best $L$ of $2L$ metrics. |
| **Memory** | $O(LN)$: $L$ copies of the LLR and partial-sum memories (plus pointers). |
| **Latency** | similar to SC per bit, plus sorting delay; grows slowly with $L$. |
| **Hardware cost** | roughly linear in $L$; $L = 8$ is a common practical choice and was widely assumed in 3GPP performance evaluations. |

**Advanced • LLR-based versus probability-based SCL**

Tal and Vardy's original decoder propagated pairs of probabilities and needed careful normalization. The LLR-based form above (Balatsoukas-Stimming et al., 2015) is numerically

stable, uses the same $f/g$ hardware as SC and needs only one extra adder per path for the metric. It is the standard in practice.

## 7.8 Chapter summary

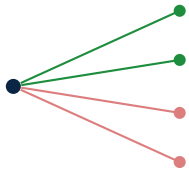
### Key concepts

- SC's commit-at-once policy causes error propagation; SCL keeps up to $L$ candidate paths.
- Information bits split paths; frozen bits do not split but can penalise wrong paths.
- Path metric: add $|L|$ when the decision disagrees with the LLR; keep the $L$ smallest metrics.
- With large $L$, SCL approaches ML decoding of the polar code.
- Complexity $O(LN \log N)$, memory $O(LN)$.

### Important formulas

| | |
|---|---|
| Exact metric | $\text{PM} \leftarrow \text{PM} + \ln(1 + e^{-(1-2\hat{u}_i)L_i})$ |
| Approximate metric | $\text{PM} \leftarrow \text{PM} + \lvert L_i \rvert \cdot [\hat{u}_i \neq \frac{1}{2}(1 - \operatorname{sign} L_i)]$ |
| Pruning | keep $L$ smallest of $\leq 2L$ candidates |

**Visual recap • Chapter 7**




penalty $= |L|$ when a decision disagrees with the LLR
frozen bits punish wrong paths
$L$ large $\Rightarrow$ ML $\Rightarrow$ limited by $d_{\min}$ $\Rightarrow$ add CRC

### Common student mistakes

**! Common mistake**

- Not penalising frozen bits. Without the frozen-bit penalty the list decoder loses most of its power.
- Sorting by *largest* metric. With the $-\ln P$ definition, small is good.
- Forgetting to copy the partial sums together with the LLRs when a path is duplicated.
- Feeding unscaled $y$ values. With min-sum $f$ and the approximate metric a common positive scale does not change any decision, but the exact $f$, the exact metric $\ln(1+e^x)$ and fixed-point hardware (quantization ranges) all need correctly scaled LLRs ($2/\sigma^2$).

### Practice problems

**Problem 7.1.** A path has PM $= 1.2$ and reaches an information bit with $L = -0.7$. Give the metrics of its two children.

**Solution** $\hat{u} = 1$ agrees with $L < 0$: 1.2. $\hat{u} = 0$ disagrees: $1.2 + 0.7 = 1.9$.

**Problem 7.2.** With $L = 2$, the four candidates after a split have metrics 3.1, 0.4, 2.2, 0.9. Which survive?

**Solution** 0.4 and 0.9.

**Problem 7.3.** Using the note on correlation discrepancy, compute the final approximate PM of codeword 11111111 for the LLRs of Example 7.1.

**Solution** Codeword bit 1 disagrees where $\alpha_j > 0$: positions $1, 4, 6$ with $0.3 + 1.0 + 3.2 = 4.5$.

## Self-assessment quiz

1. What does SCL with $L = 1$ reduce to?
2. Do frozen bits split paths?
3. Why does going from $L = 4$ to $L = 8$ give little gain in Fig. 7.2?
4. What is the memory order of SCL?

Answers: 1) SC. 2) No, but they add penalties. 3) SCL is already near ML; the code's distance limits performance. 4) $O(LN)$.

CHAPTER

# 8



# CRC-Aided SCL Decoding

**IN THIS CHAPTER YOU WILL LEARN**

- what a cyclic redundancy check (CRC) is;
- how to compute CRC bits by polynomial division;
- why a CRC makes list decoding so much stronger;
- where CRC bits are placed in a polar code;
- how the decoder selects the final candidate;
- a full transmitter/receiver chain and a real example;
- the price: rate loss and reduced error detection.

## 8.1 What is a CRC?

**Intuition**

A CRC is a short "fingerprint" of a message. The transmitter computes it and sends it with the message. The receiver recomputes the fingerprint from what it received; if the two do not match, something went wrong. Bank account numbers and ISBN numbers use the same idea with a single check digit; a CRC uses $r$ check bits and catches almost all errors.

**Definition 8.1 • Cyclic redundancy check**

Write the $A$-bit message as a polynomial $m(D) = m_0 D^{A-1} + \cdots + m_{A-1}$ over GF(2). For a fixed **generator polynomial** $g(D)$ of degree $r$, the CRC bits are the coefficients of the remainder

$$p(D) = \left(m(D)\, D^r\right) \bmod g(D).$$

The transmitted word $[\mathbf{m}\,|\,\mathbf{p}]$ corresponds to $m(D)D^r + p(D)$, which is divisible by $g(D)$. The receiver accepts a word if its division by $g(D)$ leaves remainder 0.

**Worked Example 8.1 • CRC by long division**

Message $\mathbf{m} = 1101$, generator $g(D) = D^3 + D + 1$ (1011), $r = 3$. Append three zeros and divide (XOR, no carries):

$$\begin{array}{r} 1101000 \\ \oplus\ 1011 \\ \hline 0110000 \\ \oplus\ \ 1011 \\ \hline 0011100 \\ \oplus\ \ \ 1011 \\ \hline 0001010 \\ \oplus\ \ \ \ 1011 \\ \hline 0000001 \end{array}$$

Remainder $\mathbf{p} = 001$; transmitted word 1101 001. Check: $1101001 \div 1011$ leaves remainder 000. If any single bit is flipped the remainder is non-zero.

### 8.1.1 CRC polynomials used with polar codes in 5G NR

| Name | $g(D)$ (TS 38.212, clause 5.1) | Used for |
|---|---|---|
| CRC6 | $D^6 + D^5 + 1$ | UCI, $12 \le A \le 19$ bits |
| CRC11 | $D^{11} + D^{10} + D^9 + D^5 + 1$ | UCI, $A \ge 20$ bits |
| CRC24C | $D^{24} + D^{23} + D^{21} + D^{20} + D^{17} + D^{15} + D^{13} + D^{12} + D^8 + D^4 + D^2 + D + 1$ | DCI and PBCH |

### 8.1.2 Properties

- A CRC is a *linear* code: $\mathbf{p} = \mathbf{mP}$ for an $A \times r$ binary matrix $\mathbf{P}$. Our Python class builds $\mathbf{P}$ once and then computes all CRCs with one matrix product.
- It detects every single error, every burst of length $\le r$, and (if $(D+1)$ divides $g(D)$) every odd number of errors.
- For a random wrong word, the probability that it passes the check is about $2^{-r}$.

## 8.2 How a CRC helps polar decoding

**Intuition**

In Chapter 7 the list decoder often *had* the correct message in its list — just not at the top. The path metric alone cannot always tell which candidate is right. A CRC is an independent judge: among the $L$ final candidates, choose the best one that also has a correct fingerprint.

Seen as a code, the CRC and the polar code form a **concatenated code**: outer CRC, inner polar. The concatenation has a much larger minimum distance than the polar code alone, and the list decoder is an efficient (approximate) ML decoder for the concatenation. This is why CA-SCL with moderate $L$ beats SCL with any $L$.

## 8.3 CRC attachment

1. The $A$ message bits get $r$ CRC bits: payload length $K = A + r$.
2. The $K$ payload bits are placed on the $K$ most reliable positions $\mathcal{A}$ (in index order, CRC last).
3. The rest is a normal polar encoding of length $N$.




$K = 16$ payload bits on $\mathcal{A}$ of an $N = 32$ polar code. Message rate $A/N = 0.3125$.

**! Account for the CRC in $E_b/N_0$**

The CRC bits carry no user information. A fair comparison uses the *message* rate $R = A/N$ when converting $E_b/N_0$ to noise variance. In our simulations, SCL uses $K = 128$ message bits and CA-SCL uses 128 message bits $+11$ CRC bits $= 139$ payload bits, so both have $R = 1/2$.

## 8.4 CRC-based candidate selection

**Definition 8.2 • CA-SCL decoder**

Run SCL with list size $L$. At the end, sort the $L$ surviving paths by path metric. Output the first (smallest PM) path whose payload passes the CRC. If no path passes, declare a failure (or output the best path and flag it).

```
payload = u[:, :, info]   # u: (B, L, N), paths sorted by PM; payload = message + CRC
ok = crc.check(payload) & np.isfinite(PM)
pick = np.where(ok.any(axis=1), ok.argmax(axis=1), 0)   # first passing path
u_hat = u[np.arange(B), pick, :]
```

## 8.5 Complete transmitter and receiver workflow

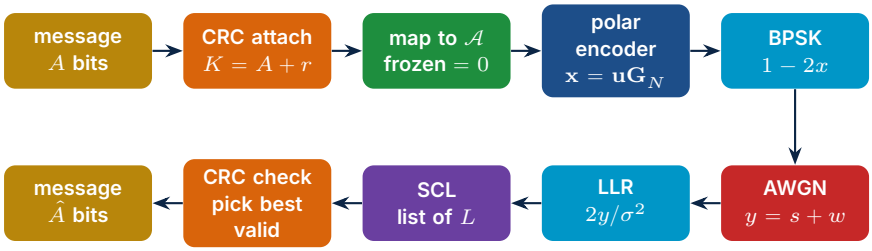



**Figure 8.1.** CRC-aided polar coding chain used in the simulations of this book.

## 8.6 Practical examples

**Worked Example 8.2 • The CRC picks the third path**

Code: $N = 32$, $A = 10$, CRC6, $K = 16$, GA construction at 2 dB: $\mathcal{A} = \{11, 13, 14, 15, 19, 21, \ldots, 31\}$. Message 1011001001; CRC6 bits 011011. One noisy reception at $E_b/N_0 = 1$ dB (LLRs rounded to one decimal) gives this final list ($L = 4$):

| rank | payload (message \| CRC) | PM | CRC | correct? |
|---|---|---|---|---|
| 1 | `1011001000 \| 010111` | 5.6 | fail | no |
| 2 | `0000100100 \| 100100` | 6.3 | fail | no |
| 3 | `1011001001 \| 011011` | 7.7 | pass | **yes** |
| 4 | `0111011010 \| 100011` | 8.0 | fail | no |

Plain SCL would output rank 1 (one message bit wrong). CA-SCL skips the two paths that fail the CRC and outputs rank 3 — the transmitted message. (Produced with the library of Chapter 11, random seed 11.)

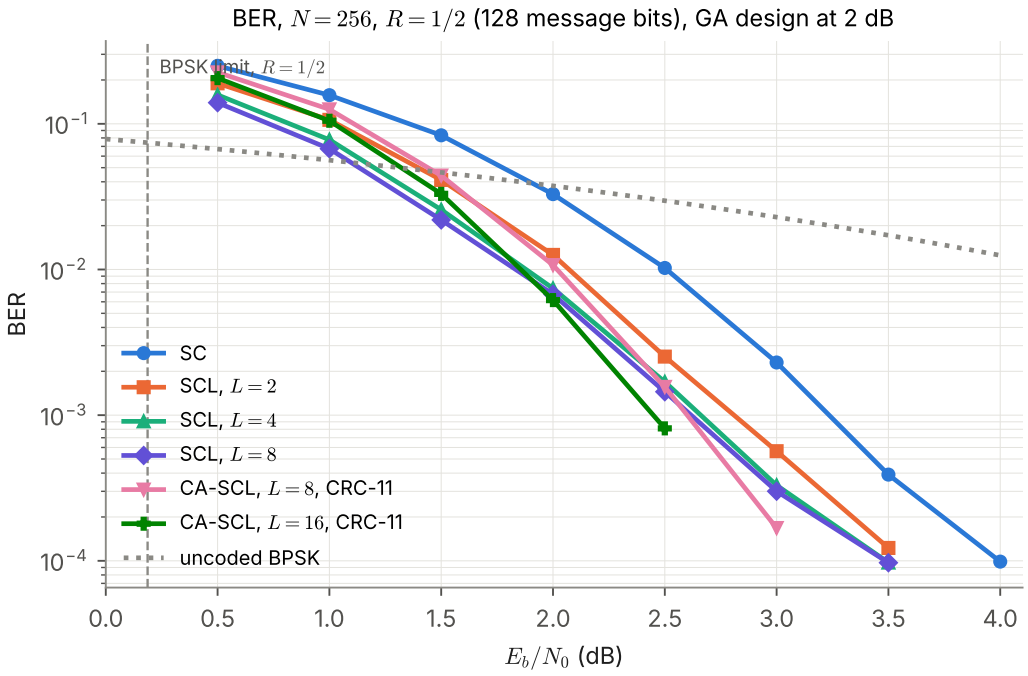



**Figure 8.2.** BER for the same simulation as Fig. 7.2. CA-SCL with CRC-11 gives the steepest curves.

**Worked Example 8.3 • Reading the simulation results**

$E_b/N_0$ needed to reach a given BLER, $N = 256$, $R = 1/2$ (Fig. 7.2, log-linear interpolation):

| Decoder | BLER $10^{-1}$ | BLER $10^{-2}$ | BLER $10^{-3}$ |
|---|---|---|---|
| SC | 2.13 dB | 3.07 dB | 3.87 dB |
| SCL, $L = 8$ | 1.54 dB | 2.56 dB | $> 3.5$ dB |
| CA-SCL, $L = 8$, CRC11 | 1.64 dB | 2.37 dB | 2.91 dB |
| CA-SCL, $L = 16$, CRC11 | 1.51 dB | 2.19 dB | $> 2.5$ dB |

- At high BLER ($10^{-1}$) the CRC costs a little: 11 extra payload bits raise the inner code rate from 0.500 to 0.543.
- At BLER $10^{-3}$ the CRC gains about 1 dB over SC, and the curve is much steeper than plain SCL.
- Doubling the list from 8 to 16 with the CRC still helps ($\approx 0.2$ dB at $10^{-2}$), unlike without CRC.

## 8.7 The price of the CRC

1. **Rate loss.** $r$ payload bits carry no data. For small blocks this matters: CRC24 on a 40-bit DCI is a large overhead. 5G therefore uses short CRCs (6, 11 bits) for small UCI.
2. **Less error detection.** The receiver uses the CRC both to *choose* among $L$ paths and to *detect* errors. With $L$ candidates the chance that some wrong path passes is about $L \cdot 2^{-r}$, i.e. roughly $\log_2 L$ CRC bits are "spent" on selection. With CRC24 and $L = 8$, about 21 bits of protection remain.

**Distributed CRC in 5G downlink**

In 5G DCI the CRC bits are *interleaved* among the information bits so that each CRC bit is placed right after the information bits it depends on. The decoder can then check CRC bits *during* decoding and kill wrong paths early (early termination), which saves power when a UE blindly tries many PDCCH candidates (Chapter 10).

## 8.8 Chapter summary

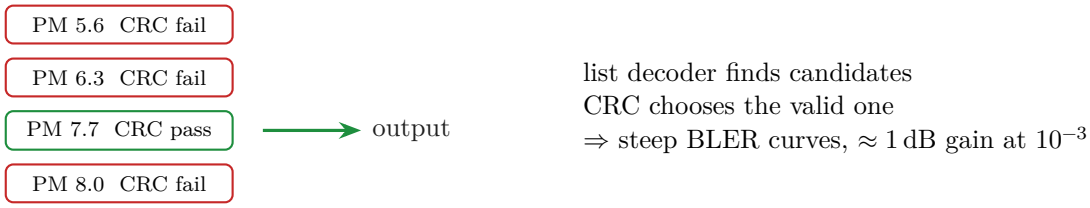
### Key concepts

- A CRC appends the remainder of $m(D)D^r$ divided by $g(D)$; it is a linear code.
- CA-SCL selects the best path that passes the CRC; this effectively decodes a concatenated code with large distance.
- Use the message rate $A/N$ for $E_b/N_0$; CRC bits are overhead.
- The CRC gives large gains at low BLER and makes larger lists useful.

## Important formulas

| | |
|---|---|
| CRC bits | $p(D) = m(D)D^r \bmod g(D)$ |
| Payload | $K = A + r$, message rate $R = A/N$ |
| False pass with a list | $\approx L \cdot 2^{-r}$ |

**Visual recap • Chapter 8**




## Common student mistakes

**! Common mistake**

- Comparing CA-SCL and SCL at the same *payload* rate $K/N$ instead of the same message rate $A/N$.
- Initialising the CRC register differently at transmitter and receiver (5G DCI starts with 24 ones prepended; our simple examples start from zero).
- Assuming CRC-$r$ still detects errors with probability $1 - 2^{-r}$ after it has been used for list selection.

## Practice problems

**Problem 8.1.** Compute the CRC of $\mathbf{m} = 1011$ with $g(D) = D^3 + D + 1$.

**Solution** $1011000 \oplus 1011000 = 0000000$: remainder 000. The message itself is $g(D)$, so its multiple $m(D)D^3$ is divisible by $g(D)$.

**Problem 8.2.** A DCI of $A = 40$ bits uses CRC24C and $N = 128$. What are $K$ and the message rate?

**Solution** $K = 64$, payload rate 0.5, message rate $40/128 = 0.3125$.

**Problem 8.3.** With CRC6 and $L = 8$, estimate the probability that a wrong list contains a path passing the CRC.

**Solution** $\approx 8 \cdot 2^{-6} = 0.125$ — CRC6 alone is weak for detection; this is why 5G adds parity-check bits for short UCI.

## Self-assessment quiz

1. Why does a CRC help list decoding more than SC decoding?

2. Where are the CRC bits placed in the polar input vector?
3. What does CA-SCL output when no path passes the CRC?
4. Which CRC protects 5G DCI?

Answers: 1) SC has only one candidate to check. 2) On information positions, after (or interleaved with) the message bits. 3) A failure flag or the best-PM path. 4) CRC24C.

CHAPTER

# 9



# Additional Professional Topics

**IN THIS CHAPTER YOU WILL LEARN**

- systematic polar encoding;
- belief propagation (BP) and SCAN decoding;
- Fast-SSC: decoding whole sub-trees at once;
- adaptive SCL;
- code construction: Bhattacharyya, density evolution, Gaussian approximation, polarization weight;
- puncturing, shortening, repetition and rate matching;
- finite-length behaviour and hardware aspects.

## 9.1 Systematic polar encoding

**Intuition**

In the normal (non-systematic) code, the message bits are hidden inside $\mathbf{u}$ and do not appear directly in $\mathbf{x}$. In a *systematic* code the message appears unchanged in the codeword. The block error rate is the same, but the *bit* error rate is lower, because a decoding error usually changes only a few of the visible message bits.

**Definition 9.1 • Systematic polar code (Arıkan, 2011)**

A systematic polar code uses the same codebook $\{\mathbf{u}\mathbf{G}_N : u_{\mathcal{F}} = \mathbf{0}\}$ but a different mapping from message to codeword: the message is placed in the codeword positions $\mathcal{A}$, i.e. $x_{\mathcal{A}} = \mathbf{m}$.

A simple encoder uses the butterfly twice (valid for frozen sets produced by the usual constructions):

1. put $\mathbf{m}$ on $\mathcal{A}$ and zeros on $\mathcal{F}$, and encode: $\mathbf{v} = \mathbf{u}\mathbf{G}_N$;
2. set the frozen positions of $\mathbf{v}$ to zero: $v_{\mathcal{F}} = \mathbf{0}$;
3. encode again: $\mathbf{x} = \mathbf{v}\mathbf{G}_N$.

**Worked Example 9.1 • Systematic encoding of the $(8, 4)$ code**

$\mathcal{A} = \{3, 5, 6, 7\}$, $\mathbf{m} = (1, 0, 1, 1)$.
Step 1: $\mathbf{u} = 00010011 \Rightarrow \mathbf{v} = 10100101$. Step 2: zero positions $0, 1, 2, 4$: $\mathbf{v} = 00000101$.
Step 3: $\mathbf{x} = \mathbf{v}\mathbf{G}_8 = 00110011$.
Check: $x_3, x_5, x_6, x_7 = 1, 0, 1, 1 = \mathbf{m}$ . It is a valid codeword: $\mathbf{x}\mathbf{G}_8 = 00000101$ has zeros on $\mathcal{F}$ . (We also verified this procedure for GA-designed codes with $N = 64$ and 256.)
**Decoding.** Run any decoder to get $\hat{\mathbf{u}}$, re-encode $\hat{\mathbf{x}} = \hat{\mathbf{u}}\mathbf{G}_N$, and read $\hat{x}_{\mathcal{A}}$.

## 9.2 Belief propagation (BP) decoding

BP decoding runs message passing on the butterfly graph of Fig. 5.1. Each butterfly is a small *processing element* with inputs $(a, b)$ on the left and outputs $(a \oplus b,\ b)$ on the right. Messages $L$ travel from right (channel) to left; messages $R$ travel from left (frozen side) to right.

**Definition 9.2 • BP update of one processing element**

With $L_{\text{up}}, L_{\text{low}}$ arriving from the right and $R_a, R_b$ from the left:

$$L_a = f(L_{\text{up}},\, L_{\text{low}} + R_b), \quad R_{\text{up}} = f(R_a,\, L_{\text{low}} + R_b),$$
$$L_b = f(L_{\text{up}},\, R_a) + L_{\text{low}}, \quad R_{\text{low}} = f(R_a,\, L_{\text{up}}) + R_b.$$

Initialization: channel LLRs on the right; $R = +\infty$ (a very large number) for frozen inputs and 0 for information inputs on the left. Iterate until the hard decisions satisfy $\hat{\mathbf{x}} = \hat{\mathbf{u}}\mathbf{G}_N$ or a maximum number of iterations is reached.

**Derivation in one line.** The element enforces $x_{\text{up}} = a \oplus b$ and $x_{\text{low}} = b$: each output message combines the other messages with $f$ when an XOR relation is involved and adds them when two views of the same bit are available — exactly the $f/g$ logic of Chapter 6, but without a fixed schedule.

| BP advantages | BP drawbacks |
|---|---|
| fully parallel, high throughput; soft outputs (useful for iterative receivers) | needs many iterations (tens); performance usually below SCL with CRC; larger memory |

## 9.3 SCAN decoding

The **soft cancellation** (SCAN) decoder of Fayyaz and Barry (2014) keeps the SC schedule (left to right through the tree) but, like BP, sends *soft* messages back instead of hard partial sums. One or two iterations already give most of the gain; it produces soft outputs with far fewer computations than BP and is attractive for turbo-equalization and iterative detection.

## 9.4 Fast-SSC decoding

**Intuition**

SC visits every leaf, even when a whole sub-tree is "easy". If a node contains only frozen bits, we already know its output: all zeros. If it contains only information bits, it is an uncoded block: just take hard decisions. Fast decoders recognise such special nodes and decode them in one step.

| Node type | frozen pattern | one-step decoding of the node's partial sums $\beta$ |
|---|---|---|
| Rate-0 | all frozen | $\beta = \mathbf{0}$ (no computation) |
| Rate-1 | all information | $\beta_j = \frac{1}{2}(1 - \operatorname{sign}\alpha_j)$ (hard decisions) |
| Repetition (Rep) | only the last bit is information | $\beta_j = b$ for all $j$, with $b = 0$ if $\sum_j \alpha_j \geq 0$, else 1 |
| Single parity check (SPC) | only the first bit is frozen | hard decisions; if their XOR is 1, flip the bit with the smallest $\lvert\alpha_j\rvert$ |

Simplified SC (SSC) by Alamdar-Yazdi and Kschischang (2011) introduced Rate-0 and Rate-1 nodes; Fast-SSC by Sarkis et al. (2014) added Rep and SPC nodes and more. The Rate-0, Rate-1 and Rep rules give exactly the SC decisions (with min-sum $f$); the SPC rule is the maximum-likelihood (Wagner) decision for the node, so the overall performance is the same as SC or marginally better — with far fewer steps.

**Worked Example 9.2 • Fast-SSC on the $(8, 4)$ code**

Frozen pattern: $u_0..u_3 =$ F F F I (a **Rep** node), $u_4..u_7 =$ F I I I (an **SPC** node). The whole code is two special nodes!
LLRs of Worked Example 6.3. Root $f$: $\alpha_L = (0.6, -2.0, -0.8, -1.2)$.
**Rep node:** $\sum \alpha_j = 0.6 - 2.0 - 0.8 - 1.2 = -3.4 < 0 \Rightarrow \beta_L = (1, 1, 1, 1)$ (so $\hat{u}_3 = 1$).
Root $g$: $\alpha_R = (0.9, -4.5, 2.6, -4.2)$.
**SPC node:** hard decisions $(0, 1, 0, 1)$, XOR $= 0$, nothing to flip: $\beta_R = (0, 1, 0, 1)$.
Output $\hat{\mathbf{x}} = [\beta_L \oplus \beta_R \,|\, \beta_R] = (1, 0, 1, 0, 0, 1, 0, 1)$, and $\hat{\mathbf{u}} = \hat{\mathbf{x}}\mathbf{G}_8 = (0, 0, 0, 1, 0, 0, 1, 1)$ — identical to the 16-step SC decoding of Chapter 6, in 4 steps.

## 9.5 Adaptive SCL decoding

Most received blocks at operating SNR are easy. Li, Shen and Tse (2012) proposed: decode with a small list (even $L = 1$); if the CRC fails, decode again with $2L$; repeat until the CRC passes or $L_{\max}$ is reached. The error rate is that of CA-SCL with $L_{\max}$, while the *average* complexity at high SNR is close to that of SC. The cost is a variable (worst-case large) latency, which matters for real-time systems.

## 9.6 Polar-code construction methods

**Definition 9.3 • Code construction**

**Construction** is the choice of the information set $\mathcal{A}$: rank the $N$ bit-channels by reliability for a given channel (or design SNR) and take the $K$ best.

| Method | Idea | Cost / accuracy |
|---|---|---|
| **Bhattacharyya (BEC)** | treat the channel as a BEC with $\epsilon = Z(W)$ and use $2Z - Z^2$, $Z^2$ | $O(N)$; approximate |
| **Monte-Carlo** | simulate SC with a genie and count errors per bit-channel | slow; accurate at the simulated SNR |
| **Density evolution (DE)** | track the full LLR density through $f$ and $g$ (Mori–Tanaka 2009) | heavy convolutions; exact in principle |
| **Tal–Vardy** | quantized DE with upgraded/degraded channels; gives bounds | $O(N\mu^2 \log \mu)$; very accurate |
| **Gaussian approximation (GA)** | LLR densities assumed Gaussian; track only the mean (Trifonov 2012) | $O(N)$; very good for AWGN |
| **Polarization weight (PW)** | SNR-independent formula $W_i = \sum_j b_j \beta^j$, $\beta = 2^{1/4}$ | $O(N \log N)$; nested; close to 5G |
| **Fixed sequence (5G)** | one standard ordering of 0..1023, nested for all $N \le 1024$ | table look-up |

### 9.6.1 Density evolution

If the channel LLR has density $a(\ell)$ (given that 0 is sent), the $g$ node ("plus") simply adds two independent LLRs, so its density is the convolution $a * a$. The $f$ node ("minus") has a more complex density (the "check-node convolution"). Applying these $n$ times along each path gives the exact LLR density of every bit-channel, and the bit-channel error probability is $P(\ell < 0)$. In practice densities are quantized, and care is needed so that quantization errors are controlled (this is what Tal–Vardy's method does rigorously).

### 9.6.2 Gaussian approximation

For BPSK over AWGN, the channel LLR is Gaussian with mean $m$ and variance $2m$ (a "consistent" Gaussian, $m = 2/\sigma^2$). GA assumes every bit-channel LLR stays consistent Gaussian and tracks only $m$:

$$m^+ = 2m, \qquad m^- = \phi^{-1}\Big(1 - \big(1 - \phi(m)\big)^2\Big), \qquad P_e(W_N^{(i)}) \approx Q\big(\sqrt{m_i/2}\big),$$

with $\phi(x) = 1 - \frac{1}{\sqrt{4\pi x}} \int_{-\infty}^{\infty} \tanh \frac{u}{2}\, e^{-\frac{(u-x)^2}{4x}} du$ for $x > 0$, $\phi(0) = 1$. A popular closed-form approximation (Chung et al.) is $\phi(x) \approx e^{-0.4527 x^{0.86} + 0.0218}$ for $0 < x < 10$ and $\phi(x) \approx \sqrt{\pi/x}\, e^{-x/4}(1 - \frac{10}{7x})$

for $x \geq 10$.

**! Numerical traps in GA**

The closed-form $\phi$ for small $x$ is slightly larger than 1, and for large $x$ the value $1-(1-\phi)^2$ rounds to 0 in double precision. Both errors silently produce wrong rankings for long codes (we observed a BLER more than four times too high at $N = 1024$ before fixing them). Our implementation evaluates $\phi$ by numerical integration for $x < 10$, works with $\ln \phi$, and uses $1-(1-p)^2 = 2p - p^2$. See `_log_phi` and `_ga_minus` in Chapter 11.

**Worked Example 9.3 • GA for $N = 4$ at design $E_b/N_0 = 2$ dB, $R = 1/2$**

$E_s/N_0 = 0.792$, so $m_0 = 4E_s/N_0 = 3.17$.
Level 1: $m^- = \phi^{-1}(1-(1-\phi(3.17))^2) = 1.63$, $m^+ = 6.34$.
Level 2: from 1.63: $m^{--} = 0.60$, $m^{-+} = 3.26$; from 6.34: $m^{+-} = 4.27$, $m^{++} = 12.68$.
Bit-channel error estimates $Q(\sqrt{m/2})$: 0.29, 0.10, 0.072, 0.0059 for $i = 0, 1, 2, 3$. The ranking $3 > 2 > 1 > 0$ is the same as for the BEC in Chapter 4.

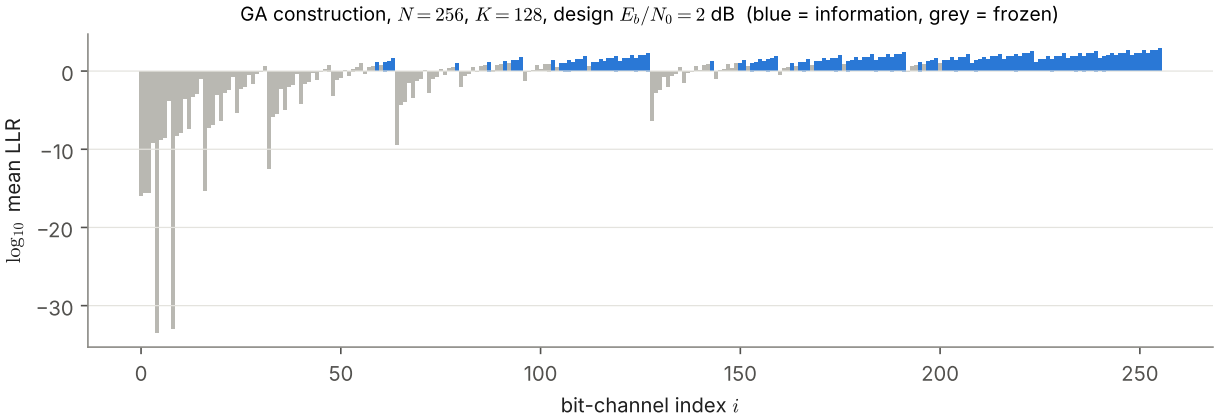



**Figure 9.1.** GA mean LLRs of the 256 bit-channels (log scale). The construction used in all our simulations.

### 9.6.3 Polarization weight

**Advanced • An SNR-independent ordering**

Write $i = \sum_{j=0}^{n-1} b_j 2^j$. The polarization weight (He et al., 2017) is $W_i = \sum_j b_j \, 2^{j/4}$; larger $W_i$ means more reliable. For $N = 16$ it gives the order (least to most reliable) $0, 1, 2, 4, 8, 3, 5, 6, 9, 10, 12, 7, 11, 13, 14, 15$; the 5G sequence restricted to $i < 16$ is $0, 1, 2, 4, 8, 3, 5, 9, 6, 10, 12, 7, 11, 13, 14, 15$ — only 6 and 9 swap. For $N = 1024$, $R = 1/2$ the two information sets share 507 of 512 positions.

## 9.7 Puncturing, shortening and repetition

Real systems need any number $E$ of transmitted bits, while the mother polar code has $N = 2^n$ bits.

**Definition 9.4 • Rate-matching operations**

- **Puncturing** ($E < N$): some coded bits are *not transmitted*. The receiver knows nothing about them: their LLR is set to 0.
- **Shortening** ($E < N$): some coded bits are forced to a value known to the receiver (zero) and not transmitted. Their LLR is set to $+\infty$ (a large number).
- **Repetition** ($E > N$): some coded bits are sent twice; the receiver *adds* their LLRs.

**Worked Example 9.4 • Which u positions must be frozen?**

Because $\mathbf{G}_N$ is lower triangular, $x_j$ depends only on $u_i$ with $i \geq j$, and $x_{N-1} = u_{N-1}$.

- **Shortening the last $k$ bits:** freeze $u_{N-k}, \ldots, u_{N-1}$. Then $x_{N-k}, \ldots, x_{N-1}$ are always 0 — known to the receiver. (These are reliable positions, so this freezing costs something; shortening is preferred at high rate.)
- **Puncturing the first $k$ bits:** the bit-channels $u_0, \ldots, u_{k-1}$ become useless (capacity 0); freeze them. They are unreliable anyway, so puncturing is cheap at low rate.

Example $N = 8$, $E = 6$: puncture $x_0, x_1$ (LLRs 0) and freeze $u_0, u_1$ — already frozen in our $(8, 4)$ code.

## 9.8 Rate matching

The full rate-matching chain (5G version, Chapter 10) is: sub-block interleaving of the $N$ coded bits, then bit selection into a circular buffer (take the first or last $E$ bits, or wrap around for repetition), and finally a channel interleaver. The choice in 5G is:

$$E \geq N: \text{ repetition;} \qquad E < N,\ \frac{K}{E} \leq \frac{7}{16}: \text{ puncturing;} \qquad E < N,\ \frac{K}{E} > \frac{7}{16}: \text{ shortening.}$$

## 9.9 Finite-length performance

| $E_b/N_0$ for BLER $= 10^{-3}$, SC, $R = 1/2$ | $N = 64$ | 128 | 256 | 512 | 1024 |
|---|---|---|---|---|---|
| simulated | 4.68 dB | 4.34 dB | 3.88 dB | 3.43 dB | 3.17 dB |
| gap to BPSK limit (0.19 dB) | 4.5 dB | 4.2 dB | 3.7 dB | 3.2 dB | 3.0 dB |

**Scaling law**

Each doubling of $N$ gains roughly 0.3–0.45 dB here, and the gain shrinks slowly. Theory predicts the gap to capacity at fixed error probability to shrink like $N^{-1/\mu}$, with a *scaling exponent* $\mu \approx 3.6$ for polar codes with SC on the BEC (Hassani, Alishahi and Urbanke), compared with $\mu = 2$ for the best possible codes. List decoding and CRC (Chapters 7–8) recover a large part of this finite-length loss — which is why 5G uses CA-SCL.

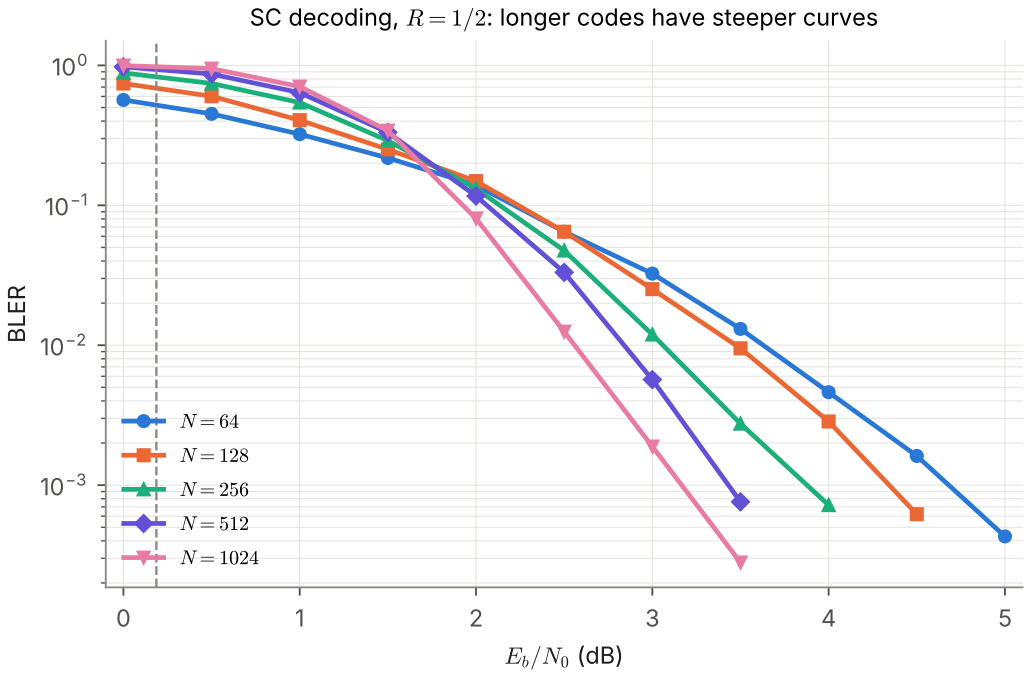



**Figure 9.2.** SC decoding at $R = 1/2$ for $N = 64$ to 1024 (GA at 2 dB). Points with fewer than 20 block errors are not shown.

## 9.10 Hardware implementation considerations

- **Quantization.** LLRs of 5–6 bits (internal 6–8 bits with saturation) lose only about 0.1 dB in typical studies. Min-sum $f$ needs only comparators and sign logic.
- **Semi-parallel SC** (Leroux et al.): $P$ processing elements shared over the tree; memory, not logic, dominates the area.
- **Latency.** Conventional SC needs $2N - 2$ cycles; Fast-SSC style node processing reduces it many times. Unrolled, fully pipelined decoders reach very high throughput for a fixed code.
- **SCL.** $L$ copies of LLR memory with pointer-based (lazy) copying; a sorter selects $L$ of $2L$ metrics each information bit. Sorter delay and memory copying are the main bottlenecks.
- **Flexibility.** 5G requires many $(K, E)$ pairs; the frozen set is computed on the fly from the reliability sequence, which favours sequence-based construction over SNR-dependent methods.

## 9.11 Chapter summary

### Key concepts

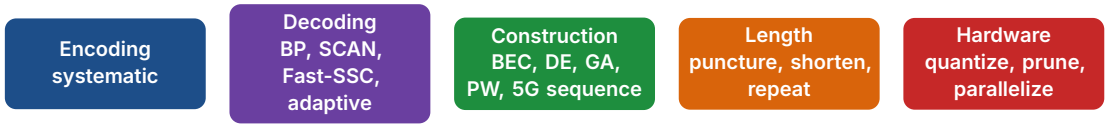
- Systematic encoding: same codebook, message visible in $x_{\mathcal{A}}$; better BER, same BLER.
- BP and SCAN give soft outputs; BP is parallel but iterative.
- Fast-SSC decodes Rate-0, Rate-1, Rep and SPC nodes in one step with identical results to SC.
- Construction ranks bit-channels: BEC bound, DE, Tal–Vardy, GA, PW, or a fixed sequence.

- Puncturing (LLR 0) suits low rates; shortening (LLR $+\infty$) suits high rates; repetition adds LLRs.

## Important formulas

| | |
|---|---|
| GA | $m^+ = 2m$, $m^- = \phi^{-1}(1-(1-\phi(m))^2)$, $P_e \approx Q(\sqrt{m/2})$ |
| PW | $W_i = \sum_j b_j 2^{j/4}$ |
| Rep node | $b = \frac{1}{2}(1 - \operatorname{sign}\sum_j \alpha_j)$ |
| 5G mode | repetition if $E \geq N$; puncture if $K/E \leq 7/16$; else shorten |

**Visual recap • Chapter 9**




## Common student mistakes

**! Common mistake**

- Setting punctured LLRs to $+\infty$ or shortened LLRs to 0 — it is the other way round.
- Using a construction designed for one $N$ after puncturing/shortening without re-freezing the affected positions.
- Believing Fast-SSC is a lossy approximation of SC. Its node rules are exact (or ML for SPC nodes).
- Comparing systematic and non-systematic codes by BLER and expecting a difference.

## Practice problems

**Problem 9.1.** Classify the nodes [F,F,F,F], [I,I,I,I], [F,F,F,I], [F,I,I,I].

**Solution** Rate-0, Rate-1, Repetition, SPC.

**Problem 9.2.** An SPC node receives $\alpha = (1.2, -0.3, -2.0, 0.8)$. Decode it.

**Solution** Hard decisions $(0,1,1,0)$, XOR $= 0 \Rightarrow$ keep: $\beta = (0,1,1,0)$. If instead $\alpha_1 = +0.3$: decisions $(0,0,1,0)$, XOR $= 1$, flip the least reliable ($|0.3|$): $(0,1,1,0)$.

**Problem 9.3.** $N = 1024$, $K = 100$, $E = 600$. Which rate-matching mode does 5G use?

**Solution** $E < N$ and $K/E = 0.167 \leq 7/16$: puncturing.

**Problem 9.4.** Compute the polarization weights of $i = 6$ and $i = 9$ for $N = 16$. Which is more reliable?

**Solution** $6 = 0110_2$: $2^{1/4}+2^{2/4} = 1.189+1.414 = 2.603$. $9 = 1001_2$: $1+2^{3/4} = 1+1.682 = 2.682$. PW ranks 9 above 6.

## Self-assessment quiz

1. Which two operations of SC are replaced by the BP processing-element equations?
2. Why does adaptive SCL have variable latency?
3. What is the mean and variance of the BI-AWGN channel LLR?
4. Why is a fixed reliability sequence attractive for standards?

Answers: 1) $f$ and $g$ (with soft instead of hard partial sums). 2) It may decode the same block several times. 3) $m = 2/\sigma^2$, variance $2m$. 4) No SNR estimate needed, nested for all lengths, cheap to store, identical at both ends.

PART IV

# Professional Level: 5G and Practical Implementation

How 3GPP NR really uses polar codes, a complete tested Python implementation, and engineering analysis of real simulation results.

CHAPTER

# 10



# Stub 10

CHAPTER

# 11



# Stub 11

CHAPTER

# 12



# Stub 12

PART V

# Expert Practice

Fully solved problems, interview and examination questions, and an end-to-end final project.

CHAPTER

# 13



# Stub 13

CHAPTER

# 14



# Stub 14

CHAPTER

# 15



# Stub 15